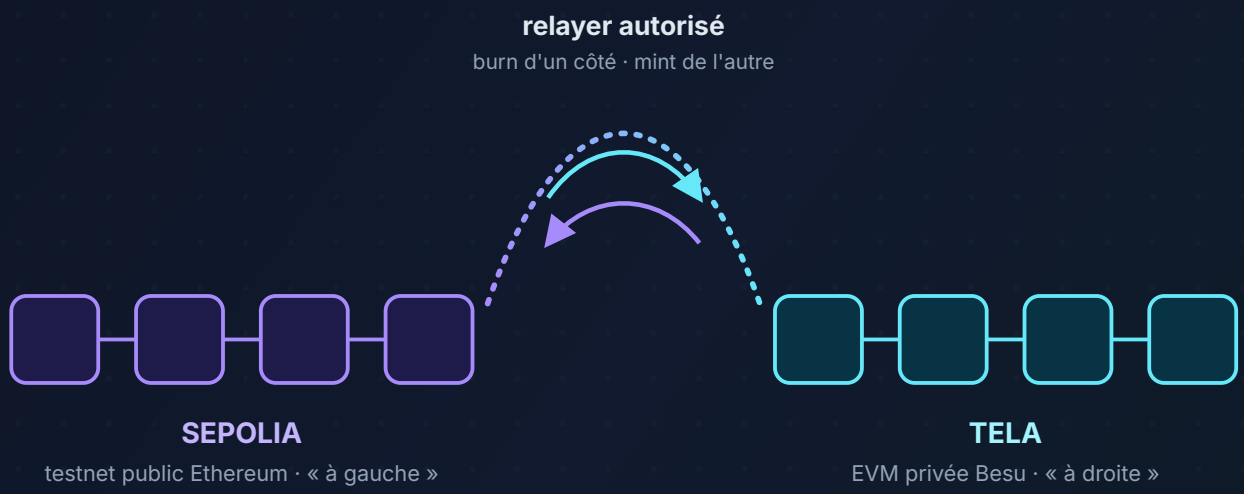
PROPOSITION · À VALIDER

PROJET TELA · TALAN ENTERPRISE LEDGER ARCHITECTURE

Bridge ERC20
Sepolia ⇄ TELA

Livrable 1 — Document d'architecture interne.
Analyse de la demande, architecture cible (infra, on-chain, off-chain, applicatif), démonstration, analyse des risques, estimation des charges et compétences requises.



Destinataire
Alain — commanditaire
Copie : Victor (référent TELA, infra et cyber)

Auteur
Matthieu Barraque
Alternant · contribution hors priorités

Version
1.0 — 30 octobre 2026
Interne Talan · diffusion restreinte

Fiche document

Objet	Livrable 1 du sujet « Bridge TELA – SEPOLIA » : document d'architecture interne expliquant comment réaliser et industrialiser un pont burn & mint pour l'ERC20 « Stablecoin Euro Talan » entre Sepolia et TELA.
Commanditaire	Alain
Référent technique	Victor — existant TELA, contraintes cyber et infrastructure Talan
Auteur	Matthieu Barraque (alternant)
Statut	PROPOSITION soumise à validation ; conditionne l'activation de l'étape 2 « Let's do it ! »
Classification	Interne Talan — diffusion restreinte. Ne contient aucun secret, aucune clé, aucune adresse IP interne.
Dépôt de référence	`m4tth1euNa0p1c/T3l4N-Bridge-ERC20` (privé), dossier `livrables/livrable-1/` ; sources du PDF et script de génération versionnés.
Données au	30 octobre 2026 (UTC). Les faits externes (statut de Sepolia, outils) sont datés et sourcés en annexe D.

Historique

Version	Date	Modification
1.0	30/10/2026	Première diffusion à Alain et Victor : analyse du mail, architecture cible, démonstration rejouée le jour même, risques, charges, compétences, feuille de route.

Comment lire ce document

Lecture rapide (10 minutes)

- **§1 Synthèse pour décision** : réponses courtes à chaque question du mail, points d'attention, recommandation.
- **§13 Feuille de route** : jalons, questions pour le rendez-vous avec Victor, décisions attendues.

Lecture complète

- §4 à §8 : architecture fonctionnelle, on-chain, off-chain, infra/réseau, composants.
- §9 : démonstration exécutée et rejouable. §10 à §12 : risques, charges, compétences.

Convention de preuve

Ce document distingue strictement ce qui est **proposé**, ce qui est **implémenté** dans le dépôt et ce qui a été **vérifié** par une exécution conservée (commande, horodatage UTC, commit, empreintes). Une vérification sur réseau local ne vaut ni pour Sepolia, ni pour le véritable TELA.

PROPOSÉ recommandation, non réalisée IMPLÉMENTÉ code présent dans le dépôt

VÉRIFIÉ LOCAL exécution conservée sur réseaux locaux HISTORIQUE preuve d'un commit antérieur, à rejouer

BLOQUÉ dépend d'une information ou d'un accès externe

Sommaire

1 Synthèse pour décision 5

1.1 L'essentiel en une page 5
1.2 Réponses directes aux questions du mail 5
1.3 Points d'attention majeurs 6
1.4 Décisions attendues 7

2 Analyse de la demande 8

2.1 Le mail d'Alain, décomposé 8
2.2 Traçabilité : ce qui est demandé et où c'est traité 8
2.3 Des contraintes aux exigences d'architecture 9
2.4 Hypothèses et inconnues 10

3 Point de départ : le démonstrateur existant 11

3.1 Ce qui existe déjà 11
3.2 Ce qui a été démontré, et avec quelle portée 12
3.3 Ce qui sépare le démonstrateur d'un pont Sepolia ⇄ TELA 12

4 Architecture fonctionnelle 13

4.1 Vue de contexte 13
4.2 Cas d'utilisation (UML) 14
4.3 Principe du burn & mint 14
4.4 Invariant comptable et réconciliation 15
4.5 Cycle de vie d'un transfert (UML états) 15

5 Architecture on-chain : les contrats 17

5.1 Modèle de classes (UML) 17
5.2 Rôles et séparation des pouvoirs 18
5.3 Identifiant de transfert et anti-rejeu 18
5.4 Garde-fous on-chain 18
5.5 Intégrer le token « Stablecoin Euro Talan » existant 19

6 Architecture off-chain et applicative 21

6.1 Architecture logique 21
6.2 Comment être notifié d'un burn ? 21
6.3 Finalité : à partir de quand peut-on minter ? 23
6.4 Séquence nominale : 100 tokens de TELA vers Sepolia 24
6.5 Reprise après incident : jamais deux mints 25
6.6 Signature et gestion des clés 26
6.7 Persistance et modèle de données 27
6.8 Supervision, alertes et exploitation 27
6.9 Répartition on-chain / off-chain 28
6.10 Alternatives étudiées 29

7 Infrastructure et réseau 30

7.1 Faut-il un nœud Sepolia chez nous ? 30
7.2 Schéma d'infrastructure cible et flux réseau 32
7.3 Matrice des flux réseau 33
7.4 Impact réseau 34
7.5 Dimensionnement 34

7.6 Environnements 34
7.7 Exigences cyber à valider avec Victor 35
8 Composants logiciels requis 36
8.1 Inventaire 36
8.2 Interfaces 36
8.3 Pile technique et versions 37
9 Démonstration 38
9.1 Démonstration rejouée le 30 octobre 2026 38
9.2 Campagnes complètes du démonstrateur (25 octobre 2026) 40
9.3 Scénario de démonstration proposé à Alain (20 minutes) 40
10 Analyse des risques projet 42
10.1 Méthode 42
10.2 Cartographie 42
10.3 Registre des risques 42
10.4 Modèle de menaces (synthèse) 43
11 Estimation des charges 45
11.1 Hypothèses de chiffrage 45
11.2 Charges par lot 45
11.3 Charges des autres contributeurs 46
11.4 Planning indicatif et scénarios de disponibilité 46
11.5 « Zéro budget » ne veut pas dire « zéro coût » 47
12 Compétences requises 48
12.1 Matrice des compétences 48
12.2 Profils et montée en compétence 49
13 Feuille de route et prochaines étapes 50
13.1 Jalons et critères de passage 50
13.2 Préparer le rendez-vous avec Victor 50
13.3 Décisions attendues et validation 51
A Annexes 52
A.1 Glossaire 52
A.2 Extraits de code commentés 53
A.3 Traçabilité des preuves 53
A.4 Sources et références 54

1 Synthèse pour décision

Alain demande un document d'architecture expliquant comment « bridger » l'ERC20 **Stablecoin Euro Talan** entre **Sepolia** (testnet public Ethereum) et **TELA** (blockchain privée EVM sur nœuds Besu), selon un principe de **burn & mint**, puis de l'industrialiser, sans SaaS, sans contrat, sans budget, et en dehors des priorités de l'alternance.

1.1 L'essentiel en une page

0 €
budget direct : 100 % open source auto-hébergé, testnets, infrastructure Talan existante

≈ 28 JH
palier 1 « premier pont réel » Sepolia ⇄ TELA de test (fourchette 17 – 45 JH)

≈ 57 JH
paliers 1 + 2 « industrialisé » (fourchette 33 – 90 JH), hors temps des autres équipes

29 / 29
contrôles réussis par la démonstration rejouée le 30/10/2026 (dont 27 tests Solidity)

18 / 18
contrôles par profil des campagnes complètes du 25/10 (Anvil et Besu locaux) — historiques, à rejouer

2
blocages externes : accès au véritable TELA de test et VM dédiée au projet

On ne part pas de zéro

Un démonstrateur complet existe déjà dans le dépôt : contrats Solidity burn/mint avec anti-rejeu, plafonds et pause ; relayer .NET persistant (PostgreSQL) ; signataire séparé ; supervision Prometheus/Grafana ; tests de panne. Il a exécuté des allers-retours réels entre deux chaînes locales, dont une chaîne **Besu QBFT** — la technologie de TELA. L'étape 2 consiste donc à **raccorder et durcir**, pas à tout construire.

Recommandation. Activer l'étape 2 en **deux paliers courts avec go/no-go** : (1) un premier pont réel Sepolia ⇄ TELA de test avec le modèle « relayer autorisé » existant et un accès Sepolia contrôlé, puis (2) l'industrialisation : nœud Sepolia interne, signature durcie, exploitation et revue sécurité. Un palier 3 optionnel (attestations multiples) renforcerait la confiance si un usage au-delà du test était envisagé.

1.2 Réponses directes aux questions du mail

Avons-nous besoin d'un nœud Sepolia chez nous ?	**Pas pour démarrer, oui pour industrialiser.** Le palier 1 peut lire Sepolia via un RPC public gratuit sans compte (si la cyber l'autorise), avec contrôle d'identité réseau. Mais le relayer croit ce que dit le RPC : un RPC tiers malveillant pourrait faire minter sur TELA des tokens jamais brûlés. La cible est donc un **nœud Sepolia interne Besu + Teku** (≈ 4 vCPU, 16 Go RAM, SSD ≥ 1 To, à mesurer). → §7.1
Quel impact réseau ?	**Aucun flux entrant depuis Internet.** Palier 1 : HTTPS sortant (443) via le proxy Talan vers un RPC Sepolia. Palier 2 : flux pair-à-pair sortants **TCP/UDP 30303** (Besu) et **9000** (Teku), non compatibles avec un proxy HTTP — c'est le point à instruire avec Victor. Côté TELA, flux interne relayer → RPC Besu. Douze flux sont décrits dans la matrice. → §7.3 – 7.4

Comment le mint/burn est-il fait ?	**Sur la chaîne, par deux contrats par réseau** (le token et un contrat Bridge). Burn : l'utilisateur appelle `requestTransfer`, qui brûle ses tokens et émet un événement dans la même transaction. Mint : le relayer appelle `completeTransfer` sur l'autre chaîne, qui vérifie rôle, paire, plafonds et anti-rejeu, puis minte — atomiquement. → §4.3, §5
On-chain ou off-chain ?	**Hybride.** La comptabilité (soldes, burn, mint, anti-rejeu, plafonds, pause) est **on-chain**. La décision de minter — « j'ai vu un burn final sur l'autre chaîne » — est prise **off-chain** par un relayer autorisé : c'est un point de confiance assumé, encadré par des plafonds et une réconciliation de l'offre, et améliorable par des attestations multiples. → §6.9, §6.10
Comment même être notifié ?	Par l'**événement** `TransferInitiated` émis lors du burn. Le relayer le lit par **interrogation historique** `eth_getLogs` **avec un point de reprise durable** : aucun événement n'est perdu après une coupure. Un abonnement WebSocket peut accélérer la détection, jamais la remplacer. On ne minte qu'après **finalité** : tag `finalized` sur Sepolia (≈ 13 min), finalité QBFT immédiate sur TELA (à confirmer). Humains : API de statut, Grafana, alertes. → §6.2 – 6.3
Quels composants logiciels ?	Sept briques, toutes open source : **contrats** (Solidity/OpenZeppelin), **relayer** (.NET/Nethereum), **signataire** séparé (puis Web3Signer), **PostgreSQL**, **accès aux nœuds** (RPC TELA, nœud Sepolia), **supervision** (Prometheus, Grafana, Alertmanager), et en option une **IHM** de suivi. Cinq existent déjà. → §8
Risques, charges, compétences ?	18 risques cotés ; les plus critiques : disponibilité du porteur, flux réseau refusés, compromission de la clé du relayer, RPC tiers non fiable, pérennité de Sepolia. Charges : ≈ 28 JH (palier 1) + ≈ 29 JH (palier 2), soit ≈ 6 à 7 mois par palier à 1 jour/semaine. Compétences : Solidity sécurité, .NET, exploitation de nœuds Ethereum/Besu, réseau, gestion de clés. → §10 – 12

1.3 Points d'attention majeurs

① Pérennité de Sepolia

L'Ethereum Foundation avait annoncé en mars 2025 une fin de vie de Sepolia prévue au **30 septembre 2026**, avec un réseau successeur. Au 30 octobre 2026, Sepolia reste actif : la presse spécialisée rapporte son passage à la mise à niveau *Glamsterdam* le 6 octobre 2026, et aucun successeur officiel n'a pu être confirmé lors de nos recherches. **Décision à prendre au cadrage** : maintenir Sepolia ou viser son successeur. L'architecture proposée est indépendante du testnet (identité réseau en configuration).

② Le relayer est un tiers de confiance

Avec le modèle burn & mint « relayer autorisé », la chaîne de destination ne vérifie pas cryptographiquement le burn. Une clé d'exécution compromise peut minter sans burn, dans la limite des plafonds. La démonstration le prouve volontairement (scénario S8) et montre que la réconciliation détecte l'écart. Acceptable pour des tokens de test ; à renforcer pour tout autre usage.

③ Le token « Stablecoin Euro Talan » existant

Le burn/mint suppose que le bridge puisse brûler et minter ce token. Son ABI, ses rôles, ses decimals et son éventuelle évolutivité (proxy) sont inconnus à ce jour. Trois options sont décrites (§5.5) ; la plus simple est d'accorder un rôle de mint/burn au contrat Bridge.

④ Réglementaire : uniquement du test

Un « stablecoin euro » réel relèverait du règlement européen MiCA (jetons de monnaie électronique), avec agrément. Ce projet se limite à des tokens **sans valeur sur réseaux de test** ; toute idée de production réelle exigerait une analyse juridique préalable, hors du périmètre de ce livrable.

1.4 Décisions attendues

#	Décision	Proposition	Qui
D1	Activer l'étape 2 sur le périmètre du palier 1 uniquement, avec go/no-go avant le palier 2	Oui	Alain
D2	Accepter le modèle « relayer autorisé » pour des tokens de test	Oui, plafonds bas	Alain, cyber
D3	Testnet public cible : Sepolia ou son successeur	Trancher au cadrage	Alain, Victor
D4	Mode d'accès Sepolia au palier 1 (RPC public sans compte ou nœud interne d'emblée)	RPC public contrôlé	Victor, cyber
D5	Disponibilité réaliste du porteur (JH/semaine) pour fixer le calendrier	1 JH/semaine	Alain
D6	Désigner un relecteur indépendant (Solidity/.NET) pour le palier 2	Un pair Talan	Alain

2 Analyse de la demande

2.1 Le mail d'Alain, décomposé

Le mail « Bridge TELA – SEPOLIA » (Victor en copie) propose de participer au projet TELA sur un sujet d'étude pouvant aller jusqu'à une mise en production. Ses éléments structurants sont les suivants.

Élément	Ce que dit le mail	Conséquence pour l'architecture
À DROITE **TELA**	« Blockchain privée EVM reposant sur des nœuds Besu et déployée dans notre infra. On peut te donner un accès au réseau et le droit de déployer des smartcontracts. »	Contrats Solidity standards ; accès RPC interne ; droits de déploiement et comptes à obtenir (TELA-01 à 06).
À GAUCHE **Sepolia**	« Réseau public. Tu pourras trouver des faucets pour te créer des ETH, et donc interagir avec la blockchain. »	Gas de test gratuit mais limité ; nécessité d'un accès à un nœud Sepolia (RPC ou nœud propre).
Mission	« Définir une infra pour bridger des tokens. » ERC20 « Stablecoin Euro Talan » déployé sur les deux chaînes.	Token existant à analyser : droits de mint/burn, decimals, évolutivité.
Principe	« Mint & burn. S'il bridge 100 tokens de TELA vers Sepolia, on burn 100 tokens côté TELA, on les mint côté Sepolia. »	Conservation de l'offre totale ; pas de verrouillage de collatéral ; anti-rejeu indispensable.
Ambition	« On veut industrialiser cela. »	Reprise sur incident, supervision, exploitation, sécurité des clés, documentation.
Livrable 1	Document d'architecture interne : sujets infra, archi, applicatifs ; analyse de risque ; estimation des charges et compétences. Ensuite, activation probable de l'étape 2 « Let's do it ! ».	Le présent document ; il prépare un go/no-go.
Contraintes	Contraintes cyber et infra Talan (cf. Victor). Pas de SaaS ni de prestataire avec contrat ou budget. Pas de budget. Sujet non prioritaire : uniquement après le travail demandé par Talan et le client.	Open source auto-hébergé ; réutilisation de l'existant ; paliers courts ; chiffrage en jours et en calendrier.
Suite	Prendre rendez-vous avec Victor pour comprendre ce qu'est TELA.	Questionnaire préparé en §13.2.

> **Limite de l'analyse**
> Les captures reçues couvrent le début du mail (jusqu'à « ...on les mint côté SEPOLIA. ») et sa fin (à partir de « On veut industrialiser cela. »). Une éventuelle partie intermédiaire n'était pas visible : ce document ne suppose rien de son contenu et devra être complété si elle apporte des exigences supplémentaires.

2.2 Traçabilité : ce qui est demandé et où c'est traité

Réf.	Attente du livrable 1	Section	Couverture
A-01	Infra : avons-nous besoin d'un nœud Sepolia chez nous ?	§7.1, §7.5	TRAITÉ
A-02	Infra : quel impact réseau ?	§7.2 à §7.4	TRAITÉ
A-03	Archi : comment le mint/burn est fait ?	§4.3, §5, §6.4	TRAITÉ

Réf.	Attente du livrable 1	Section	Couverture
A-04	Archi : on-chain / off-chain ?	§6.9, §6.10	TRAITÉ
A-05	Archi : comment même être notifié ?	§6.2, §6.3	TRAITÉ
A-06	Applicatif : quels composants logiciels requis ?	§6.1, §8	TRAITÉ
A-07	Analyse du risque projet	§10	TRAITÉ
A-08	Estimation des charges pour la réalisation	§11	TRAITÉ
A-09	Compétences requises	§12	TRAITÉ
A-10	Contraintes cyber et infra Talan	§7.7, §13.2	PARTIEL à valider avec Victor
A-11	Préparer l'étape 2 « Let's do it ! »	§13	TRAITÉ

2.3 Des contraintes aux exigences d'architecture

Contrainte	Exigence d'architecture retenue	Effet visible
Pas de SaaS, pas de contrat	Composants open source auto-hébergeables sous licence permissive ; aucun compte chez un fournisseur de nœuds (Infura, Alchemy…) ni de protocole de bridge opéré par un tiers (CCIP, LayerZero…).	Bridge maison ; nœud Sepolia interne en cible.
Pas de budget	Réutiliser l'infrastructure Talan (VM, proxy, SMTP, supervision) ; gas Sepolia par faucets ; aucun achat de licence, de HSM ou de service.	Coût direct nul ; coût en temps chiffré (§11.5).
Cyber et infra Talan	Aucun flux entrant Internet ; cloisonnement par zones ; moindre privilège ; secrets hors code ; journaux sans secret ; versions épinglées ; validation des flux avant ouverture.	Matrice des flux, zones (§7).
Sujet non prioritaire	Paliers courts et autonomes ; automatisation (une commande pour démarrer, tester, prouver) ; documentation continue ; reprise facile après plusieurs semaines d'interruption.	Paliers, scénarios de calendrier.
Industrialiser	Reprise sans perte ni double mint ; supervision honnête (donnée périmée visible) ; procédures d'exploitation ; tests de panne ; preuves conservées.	Relayer persistant, runbooks.
Réseau privé + réseau public	Politique de finalité par réseau ; aucune hypothèse TELA déduite d'un Besu local ; identité réseau vérifiée à chaque cycle.	Profils séparés.

2.4 Hypothèses et inconnues

Hypothèses de travail

- Tokens **de test, sans valeur** ; aucun mainnet, aucun actif réel.
- Même nombre de decimals sur les deux chaînes (18), pas de frais de transfert ni de rebase.
- Une offre de référence connue ; pas d'émission hors bridge pendant les tests.
- TELA accepte le déploiement de contrats standards (London ou plus récent).
- Le bridge est opéré par Talan ; un seul relayer au palier 1.

Inconnues à lever (détail en §13.2)

- Identité du TELA de test : chain ID, hash genesis, version Besu, consensus, forks EVM.
- Accès RPC TELA : URL, TLS, authentification, VPN, ACL, permissionnement des comptes.
- Token existant : adresses, ABI, rôles, administrateur, évolutivité.
- Flux réseau autorisés, coffre de secrets, supervision et SMTP réutilisables.
- Statut de Sepolia et de son successeur à la date de démarrage.

3 Point de départ : le démonstrateur existant

Le dépôt `T3l4N-Bridge-ERC20` contient un démonstrateur de bridge ERC20 bidirectionnel, construit avant ce livrable pour éprouver le protocole. Il fixe le socle technique proposé et réduit fortement le risque de l'étape 2.

3.1 Ce qui existe déjà

Brique	Contenu	Technologie	Statut
Contrats	`BridgeToken` (ERC20 de test, 18 decimals, rôle `BRIDGE_ROLE`) et `Bridge` (burn, mint, anti-rejeu, plafonds, pause, rôles séparés). ≈ 280 lignes, 777 lignes de tests (unitaires, fuzzing, invariants).	Solidity 0.8.30, OpenZeppelin 5.4.0, Foundry	VÉRIFIÉ LOCAL
Relayer	Collecte historique avec checkpoint, contrôle d'identité réseau, machine à états à 12 états, réservation de nonces, persistance avant diffusion, reprise, réconciliation d'offre, API `/status`, `/metrics`. ≈ 1 140 lignes.	.NET 10, Nethereum 6.1, Npgsql	VÉRIFIÉ LOCAL
Signataire	Service séparé, seul à détenir les clés ; politique stricte (méthode, domaine, montant, gas, fréquence) ; jeton de service ; aucune connexion RPC. ≈ 590 lignes.	.NET 10, Nethereum.Signer	VÉRIFIÉ LOCAL
Persistance	Schéma `bridge` : réseaux, blocs, observations, transferts, nonces, tentatives, réconciliations ; compte applicatif non administrateur.	PostgreSQL 17	VÉRIFIÉ LOCAL
Supervision	16 métriques, 12 règles d'alerte, tableau de bord d'exploitation ; donnée périmée explicitement signalée.	Prometheus 3, Grafana 13	VÉRIFIÉ LOCAL
Environnements	Profils `local-fast` (deux Anvil) et `local-besu` (Besu 26.10 QBFT + Anvil) ; conteneurs durcis ; profil `integration-testnet` volontairement refusé.	Docker Compose	VÉRIFIÉ LOCAL
Preuves	Interface `bootstrap / up / demo / test / prove / status / down` ; manifestes de campagne avec empreintes ; registre de 28 affirmations ; CI GitHub Actions.	Python 3.12	IMPLÉMENTÉ
Décisions	Six ADR : stack et profils, protocole et confiance, reprise et finalité, interfaces, persistance Anvil, arrêts prolongés.	MkDocs	IMPLÉMENTÉ

3.2 Ce qui a été démontré, et avec quelle portée

Campagnes complètes du 25 octobre 2026

HISTORIQUE

Sur le commit `b97c541`, deux campagnes `prove` ont réussi **18 contrôles sur 18** chacune : `local-fast` (00:24 → 00:32 UTC) et `local-besu` (00:33 → 00:40 UTC), plus la CI sur clone neuf.

- Aller-retour réel de 10 tokens dans les deux sens, offre restaurée.
- 9 scénarios de panne sur Anvil, 7 sur Besu (relecture, pause, RPC suspendu, gas, concurrence, crashs, réorganisation).
- Cloisonnement réseau, fichiers et secrets vérifié sur 14 services.

Démonstration du 30 octobre 2026

VÉRIFIÉ LOCAL

Rejouée pour ce livrable sur le commit `2f05504` : **29 contrôles sur 29** en 13 secondes.

- 27 tests Solidity : 22 tests de protocole dont 2 fuzz (1 000 exécutions), 2 tests de domaine, 2 invariants (256 séquences × 64 appels) et 1 test d'atteignabilité.
- Aller-retour 100 puis 40 tokens, rejeu, rôle, plafond, pause, réconciliation.
- Mint fabriqué par l'exécuteur : accepté par le contrat, détecté par la réconciliation.

Pourquoi « historique » ?

Le 27 octobre, un arrêt des nœuds locaux (mémoire Anvil, enregistreur JFR de Besu) a conduit à des correctifs (ADR-0006). Toute modification des sources rend les preuves antérieures **obsolètes** jusqu'à une nouvelle campagne complète : le registre compte aujourd'hui 26 affirmations obsolètes et 2 bloquées. Les campagnes citées au journal du 27 octobre ne sont pas versionnées dans `evidence/` et ne sont pas reprises ici. Rejouer `prove` sur les deux profils est la première tâche technique du palier 1.

3.3 Ce qui sépare le démonstrateur d'un pont Sepolia ⇄ TELA

Écart	Traitement proposé	Palier
Profil `integration-testnet` désactivé, aucune identité Sepolia/TELA configurée	Activer le profil avec identités vérifiées (chain ID, genesis, code des contrats)	1
Finalité : conventions locales uniquement (3 blocs Anvil, tête QBFT locale)	Tag `finalized` Sepolia ; politique TELA selon consensus confirmé	1
RPC locaux en HTTP sans authentification ; client limité aux hôtes locaux	HTTPS, authentification, proxy, limites de plage `eth_getLogs`	1
Transactions « legacy » à prix de gas fixe	Transactions EIP-1559 avec plafond de frais (Sepolia)	1
Bridge limité au bytecode exact de `BridgeToken`	Adapter au token « Stablecoin Euro Talan » (§5.5)	1
Clés en fichiers de laboratoire	Keystore chiffré (P1), Web3Signer + coffre ou HSM (P2)	1 → 2
VM dédiée non fournie (hôte Windows partagé)	VM Talan dédiée et durcie	1
Pas de sauvegarde/restauration testée, pas de runbooks	Procédures d'exploitation et tests de restauration	2

4 Architecture fonctionnelle

4.1 Vue de contexte

Conformément au mail, Sepolia est représentée **à gauche** et TELA **à droite**. L'utilisateur agit directement sur les chaînes avec son portefeuille ; le service de bridge, opéré par Talan hors chaîne, observe les burns d'un côté et déclenche les mints de l'autre.

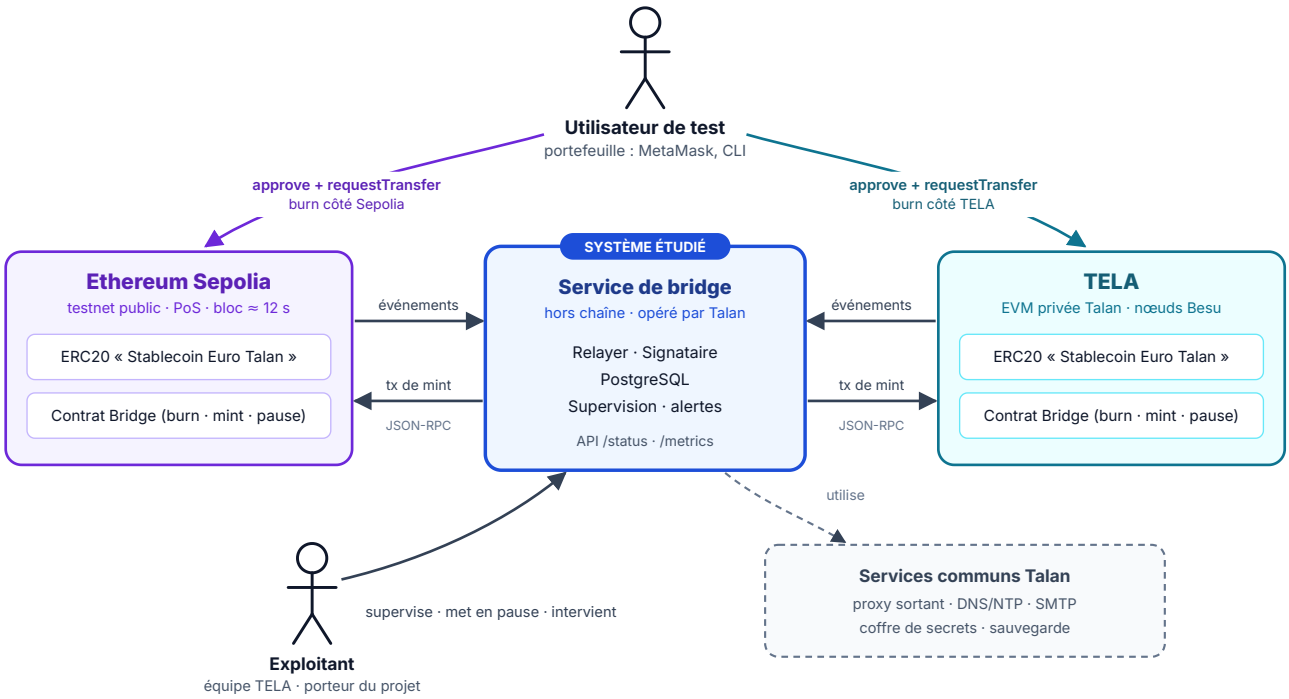


Figure 1 — Vue de contexte (niveau C4 « système »). Le service de bridge n'a aucune autorité sur les soldes : il ne peut que soumettre des mints, bornés par les contrats.

4.2 Cas d'utilisation (UML)

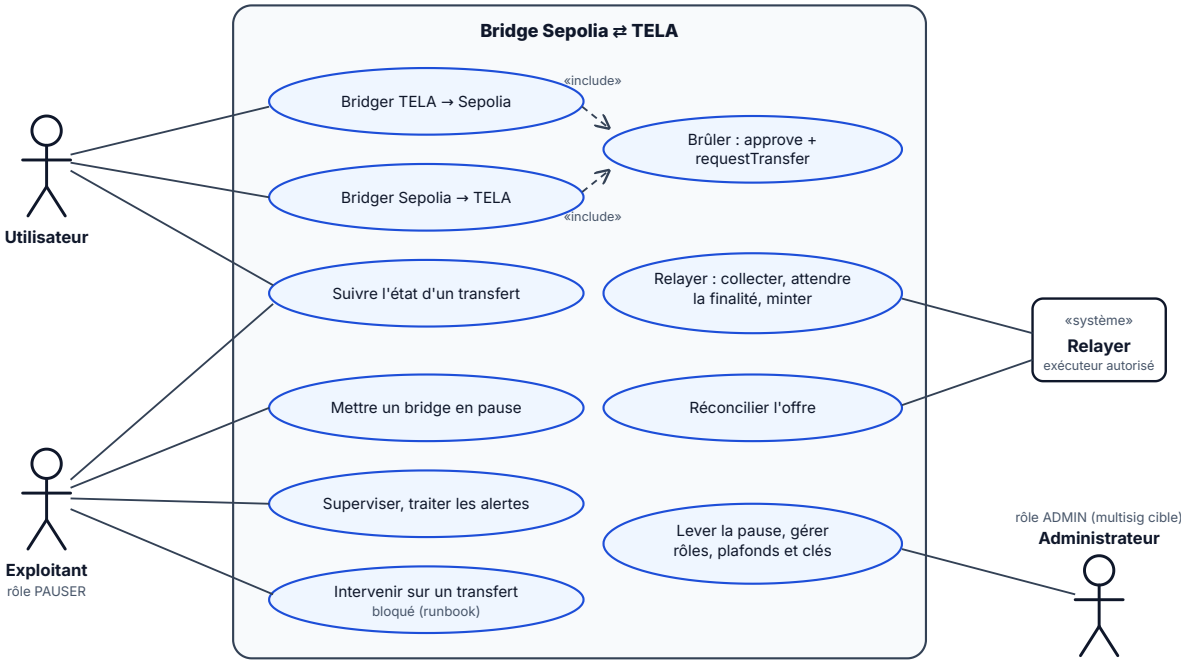


Figure 2 — Diagramme UML des cas d'utilisation. Les rôles on-chain (PAUSER, EXECUTOR, ADMIN) sont portés par des comptes distincts ; la levée de pause est réservée à l'administrateur.

Cas	Acteur principal	Préconditions → résultat
UC1/2	Utilisateur	Solde suffisant, bridge actif, montant ≤ plafond → tokens brûlés à la source, même montant crédité au destinataire après finalité.
UC3	Utilisateur	Identifiant du transfert (hash de la tx de burn) → état : observé, en attente de finalité, soumis, finalisé, en intervention.
UC4	Relayer	Événement final, identités et rôles vérifiés → un seul mint par identifiant, quels que soient pannes et redémarrages.
UC5/6	Exploitant (pauser)	Incident suspecté → `pause()` bloque burns et mints ; seul l'administrateur peut lever la pause.
UC7	Relayer / exploitant	Coupes finalisées couvertes par la collecte → `balanced`, `mismatch`, `inconclusive` ou `unavailable`.
UC8	Exploitant	Alerte (nonce bloqué, transfert ancien, écart d'offre) → procédure d'intervention documentée, jamais de mint compensatoire automatique.

4.3 Principe du burn & mint

Exemple du mail : un utilisateur bridge **100 tokens de TELA vers Sepolia**. L'offre totale est conservée : ce qui disparaît d'un côté réapparaît de l'autre, jamais deux fois.

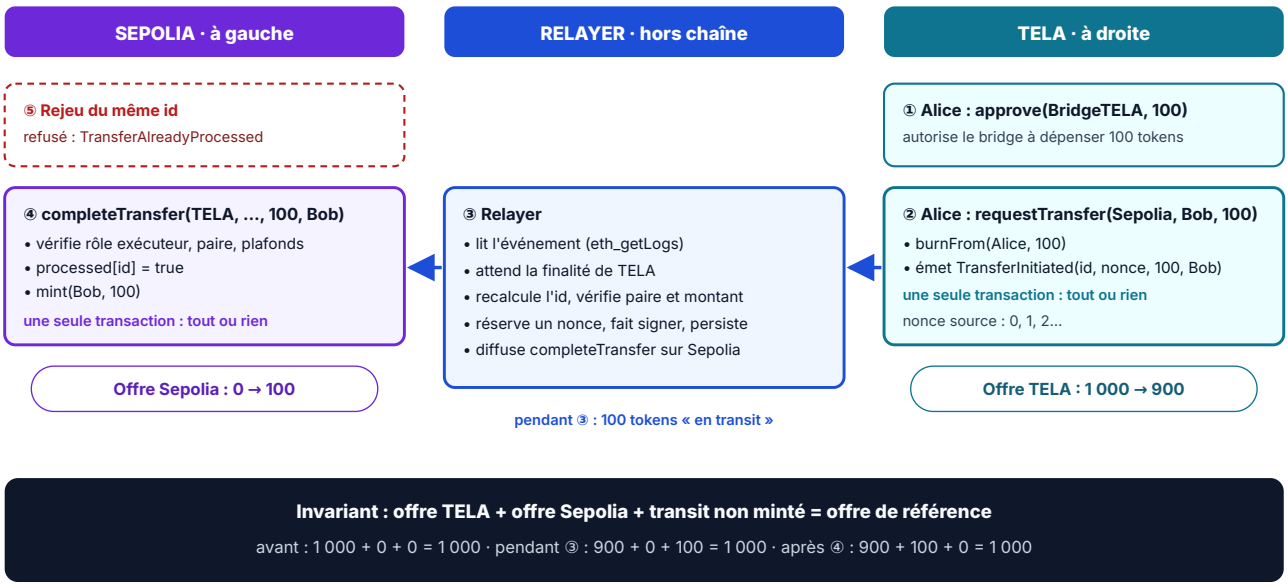


Figure 3 — Burn & mint de 100 tokens de TELA vers Sepolia. Le sens inverse est strictement symétrique. Valeurs illustratives.

4.4 Invariant comptable et réconciliation

La propriété métier centrale est la **conservation de l'offre**. La réconciliation implémentée la vérifie à des hauteurs de bloc explicites et finalisées sur chaque chaîne :

```
offre_A(coupe_A) + offre_B(coupe_B) + transit_non_minté(coupes) = offre_de_référence
// transit : burns canoniques inclus dans la coupe source sans completion dans la coupe destination,
// comptés une fois par identifiant, dans les deux sens.
```

Résultat	Signification	Action
balanced	Égalité satisfaite sous les hypothèses (pas d'émission externe)	Aucune
mismatch	Écart : mint sans burn, émission externe, bug	Alerte, enquête, pause si nécessaire
inconclusive	Coupes incohérentes (mint vu sans son burn), collecte en retard, réorganisation pendant le contrôle	Rejouer ; jamais une validation
unavailable	RPC ou base indisponibles	Rétablir, rejouer

Limite : des émissions et destructions externes qui se compensent échappent à cette seule égalité ; la réconciliation n'est pas un registre exhaustif des pouvoirs de mint. Elle détecte en revanche un mint fabriqué par l'exécuteur (démonstration, scénario S8).

4.5 Cycle de vie d'un transfert (UML états)

Chaque transfert est une ligne PostgreSQL dont l'état évolue selon la machine ci-dessous. La transaction signée est **persistée avant diffusion** : après un crash ou un timeout, le relayer rediffuse les mêmes octets (même nonce, même hash) au lieu de créer un second mint.

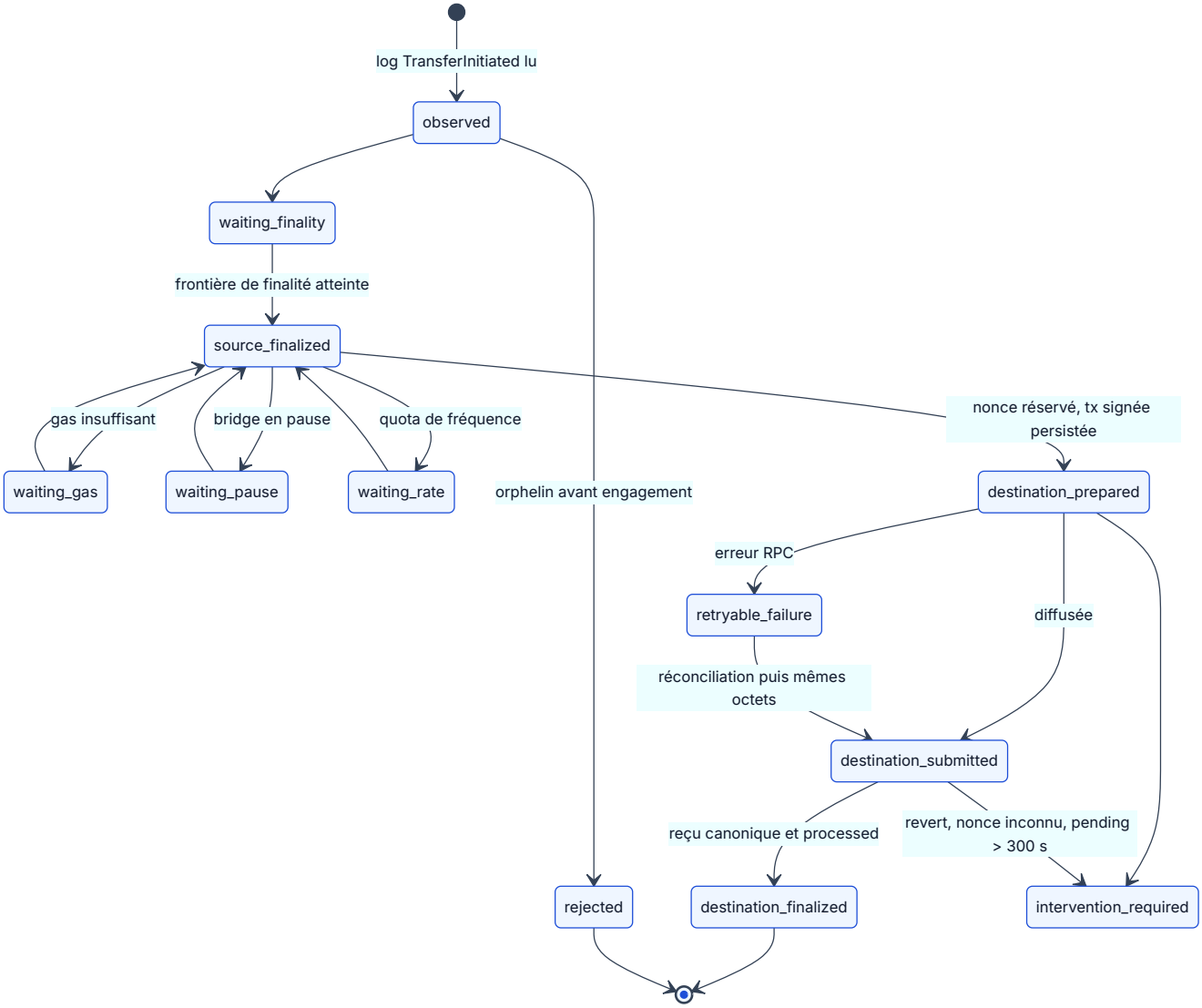


Figure 4 — Diagramme UML d'états d'un transfert (12 états, contrainte `CHECK` du schéma SQL). Les états d'attente sont réévalués à chaque cycle.

5 Architecture on-chain : les contrats

Sur chaque chaîne, deux contrats : le **token ERC20** et le **contrat Bridge**. Le Bridge est le seul à pouvoir minter et brûler ; il porte les règles de sécurité (anti-rejeu, plafonds, pause, appariement).

5.1 Modèle de classes (UML)

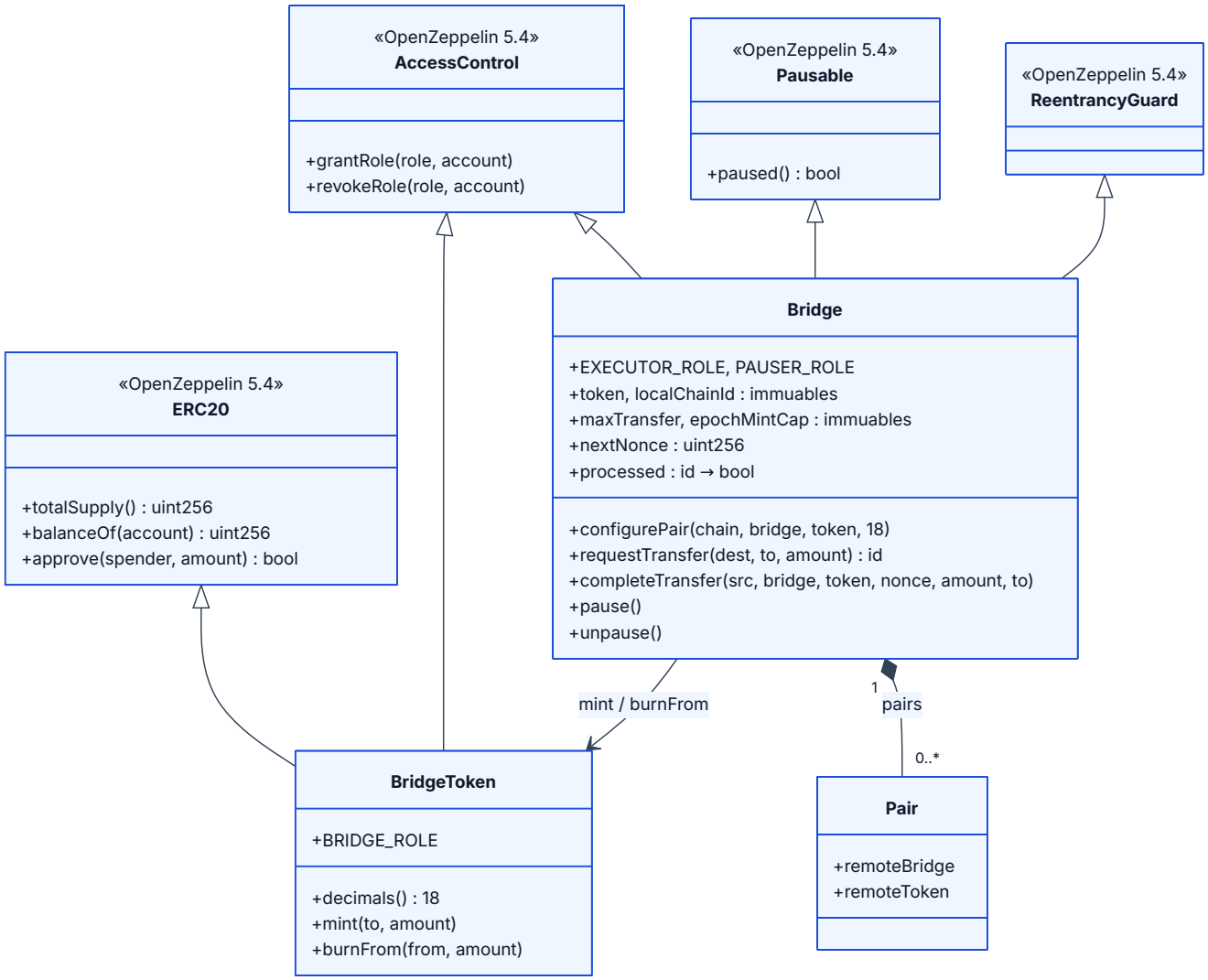


Figure 5 — Diagramme UML de classes des contrats (`contracts/src/Bridge.sol`, `BridgeToken.sol`). Rôles requis : `mint` / `burnFrom` → BRIDGE ; `completeTransfer` → EXECUTOR ; `pause` → PAUSER ; `configurePair` / `unpause` → ADMIN. Aucun proxy : le code déployé est immuable.

Événements émis

Événement	Champs	Usage
`TransferInitiated`	`transferId` (indexé), `destinationChainId` (indexé), `sender` (indexé), `recipient`, `sourceToken`, `destinationToken`, `destinationBridge`, `nonce`, `amount`	Notification du burn ; lu par le relayer
`TransferCompleted`	`transferId` (indexé), `recipient` (indexé), `amount`	Preuve du mint ; suivi utilisateur
`PairConfigured`	`remoteChainId` (indexé), `remoteBridge`, `remoteToken`	Audit de la configuration

5.2 Rôles et séparation des pouvoirs

Rôle	Pouvoir effectif	Porteur recommandé	Exposition
Admin du Bridge `DEFAULT_ADMIN_ROLE`	Configurer une paire (une seule fois), attribuer/retirer les rôles, lever la pause	Multisig 2 sur 3 (Safe, open source) en palier 2 ; clé hors ligne au palier 1	FROIDE
Pauser `PAUSER_ROLE`	Mettre en pause burns et mints ; ne peut pas lever la pause	Exploitant d'astreinte	TIÈDE
Exécuteur `EXECUTOR_ROLE`	Appeler `completeTransfer` sur la paire configurée, dans les plafonds	Signataire du relayer, une clé par chaîne	CHAUDE
Bridge du token `BRIDGE_ROLE`	`mint` et `burnFrom` (ce dernier dépense l'allowance du détenteur)	Le contrat Bridge uniquement	CONTRAT
Admin du token	Attribuer `BRIDGE_ROLE`, donc autoriser des émissions hors bridge	Même multisig que l'admin du Bridge	FROIDE

5.3 Identifiant de transfert et anti-rejeu

Chaque transfert a un identifiant unique, calculé de façon identique par le contrat source, le relayer et le contrat destination. Il est **séparé par domaine** : tout changement de chaîne, de contrat, de token, de montant ou de destinataire produit un autre identifiant, ce qui empêche de rejouer une preuve d'un contexte dans un autre.

```
transferId = keccak256(abi.encode(
    keccak256("T3L4N_BRIDGE_BURN_MINT"), uint256(1),       // domaine et version du protocole
    sourceChainId, sourceBridge, destinationChainId, destinationBridge,
    sourceToken, destinationToken, nonce, amount, recipient))       // nonce source : 0, 1, 2…
```

Sur la chaîne de destination, `processed[transferId]` passe à `true` dans la même transaction que le mint ; un second appel échoue avec `TransferAlreadyProcessed`. Un vecteur de test indépendant (`contracts/vectors/transfer-id-v1.json`) fige ce calcul ; la démonstration le recalcule hors chaîne et obtient le même résultat (§9).

5.4 Garde-fous on-chain

Limites de montant

- **Par transfert** : 1 000 tokens (paramètre immuable).
- **Par époque** : 100 000 tokens mintés par heure et par bridge de destination (fenêtre fixe).
- Montants entiers en unités minimales (10^{18}) : aucun arrondi.

Configuration

- **Paire définitive** : impossible de rediriger un burn en transit vers un autre contrat.
- Decimals imposés à 18 des deux côtés ; tokens à frais ou rebase refusés.
- Détection d'un changement de chaîne (`NetworkChanged`).

Arrêt d'urgence

- `pause()` bloque burns et mints, séparément sur chaque chaîne.
- Levée de pause réservée à l'administrateur (deux personnes distinctes).
- Un burn en transit reste mintable après la levée.

Robustesse du code

- Protection contre la réentrance sur les deux chemins métier.
- Effets atomiques : si le mint échoue, `processed` et le quota sont annulés.
- Pas de proxy ni de fonction d'administration des plafonds.

> **Ce que les contrats ne font pas**
>
> Le contrat de destination ne vérifie **aucune preuve** du burn : il fait confiance à l'exécuteur. Les plafonds limitent l'impact d'une clé compromise (au plus 100 000 tokens par heure et par sens), la pause l'arrête, la réconciliation le détecte. Ce n'est pas une garantie cryptographique.

5.5 Intégrer le token « Stablecoin Euro Talan » existant

Le démonstrateur utilise son propre token de test. Le mail indique qu'un ERC20 « Stablecoin Euro Talan » est **déjà déployé sur les deux chaînes**. Le choix de l'option dépend de ses capacités, inconnues à ce jour (TELA-04).

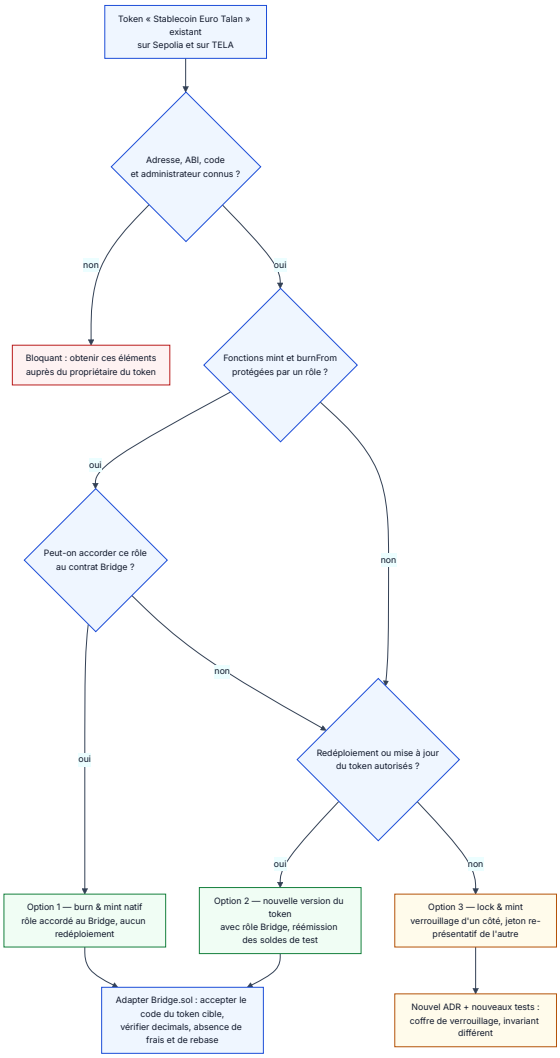


Figure 6 — Arbre de décision pour le token existant. L'option 1 est la plus simple ; l'option 3 change le modèle et l'invariant.

Option	Avantages	Inconvénients	Effort
1. Rôle au Bridge	Respecte exactement le mail ; token inchangé ; offre totale conservée	Suppose un token à rôles ; le Bridge doit accepter un autre code que `BridgeToken`	2 – 4 JH
2. Nouvelle version	Token conçu pour le bridge ; contrôle total des rôles	Nouvelles adresses ; soldes de test à réémettre ; communication aux utilisateurs	3 – 6 JH
3. Lock & mint	Aucune modification du token d'origine	S'écarte du principe burn & mint ; collatéral verrouillé à protéger ; nouveau modèle de risque	6 – 10 JH

6 Architecture off-chain et applicative

6.1 Architecture logique

Le diagramme logique ci-dessous répartit les composants en quatre couches. Les flèches indiquent qui appelle qui ; aucune couche supérieure n'accède directement aux clés.

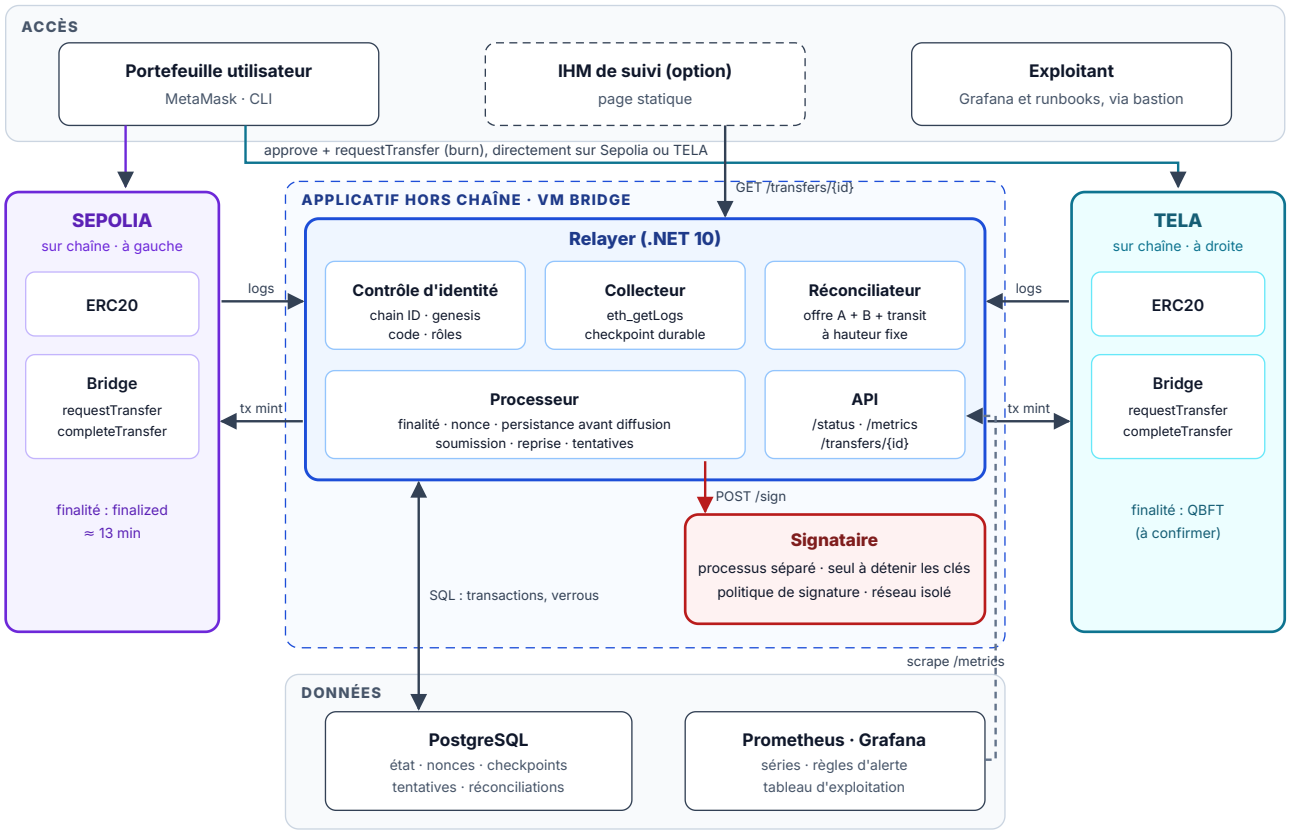


Figure 7 — Diagramme logique en couches. Le relayer regroupe identité, collecte, traitement, réconciliation et API ; le signataire est un processus séparé ; les utilisateurs brûlent directement sur la chaîne.

6.2 Comment être notifié d'un burn ?

La question du mail est centrale : un bridge qui « rate » un événement laisse des tokens brûlés sans contrepartie. Cinq mécanismes ont été comparés.

Mécanisme	Atouts	Limites	Verdict
Interrogation historique `eth_getLogs` + checkpoint durable	Aucune perte : on reprend au dernier bloc lu, même après des jours d'arrêt ; idempotent ; HTTPS simple, compatible proxy ; vérifiable (hash de bloc, nonce source)	Latence = intervalle d'interrogation ; plages de blocs limitées par certains RPC (pagination nécessaire)	RETENU source de vérité
Abonnement WebSocket `eth_subscribe`	Réactivité immédiate	Événements perdus pendant une déconnexion ; pas de rattrapage ; WebSocket difficile derrière un proxy	OPTION simple « réveil »
Filtres côté nœud `eth_newFilter`	Simple	État perdu au redémarrage du nœud, expiration des filtres	ÉCARTÉ

Mécanisme	Atouts	Limites	Verdict
Indexeur (The Graph auto-hébergé, Ponder…)	Requêtes riches, historique	Composant lourd de plus à exploiter et sécuriser	ÉCARTÉ palier 1–2
Plugin Besu côté TELA	Notification native par le nœud	Installation sur les nœuds TELA (gouvernance de Victor) ; inutile côté Sepolia	À DISCUTER

Pourquoi l'interrogation suffit

Le délai dominant est la **finalité**, pas la détection : on ne minte pas avant que le burn soit irréversible. Sur Sepolia, la finalité prend environ 13 minutes ; interroger toutes les quelques secondes ne change rien à l'expérience utilisateur, alors qu'un abonnement seul ferait courir un risque de perte.

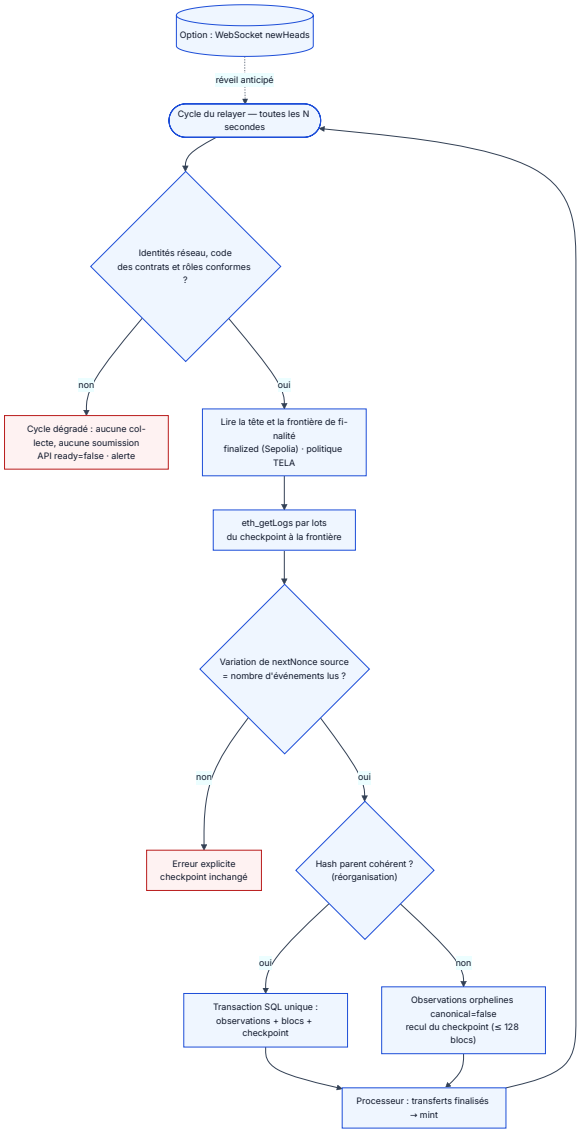


Figure 8 — Diagramme d'activité de la détection. Le checkpoint et les observations d'un lot sont commis dans la même transaction : une coupure ne peut ni perdre ni dupliquer un événement.

Et pour les humains ?

- **Utilisateur** : l'identifiant du transfert est dans l'événement de burn ; une IHM ou un script interroge `processed(id)` ou l'événement `TransferCompleted` sur la destination. Une route `GET /transfers/{id}` est prévue au palier 1.
- **Exploitant** : tableau Grafana, règles Prometheus (transfert ancien, intervention requise, écart d'offre, gas bas, donnée périmée), notification par l'Alertmanager vers la messagerie interne Talan (SMTP), sans service externe.

6.3 Finalité : à partir de quand peut-on minter ?

Réseau	Mécanisme	Politique proposée	Délai indicatif
SEPOLIA	Preuve d'enjeu Ethereum : un bloc est *finalisé* après deux époques justifiées (64 créneaux de 12 s)	Bloc ≤ tag `finalized` du client de consensus ; jamais `latest`	≈ 13 – 15 min en conditions normales
TELA	Si QBFT : finalité immédiate dès qu'un bloc est validé par plus des 2/3 des validateurs (pas de fork)	Bloc inclus, sous réserve de confirmation du consensus et de la version (TELA-03)	≈ 1 période de bloc (quelques secondes)
LOCAL	Anvil : convention de 3 blocs ; Besu local : tête QBFT mono-validateur	Tests uniquement	—

Latences de bout en bout estimées (non mesurées) : TELA → Sepolia ≈ finalité TELA + intervalle de collecte + inclusion Sepolia, soit **moins d'une minute** ; Sepolia → TELA ≈ finalité Sepolia + collecte + inclusion TELA, soit **environ 15 minutes**. Une politique inconnue ou une lecture historique impossible provoque une erreur, jamais un repli silencieux vers `latest`.

6.4 Séquence nominale : 100 tokens de TELA vers Sepolia

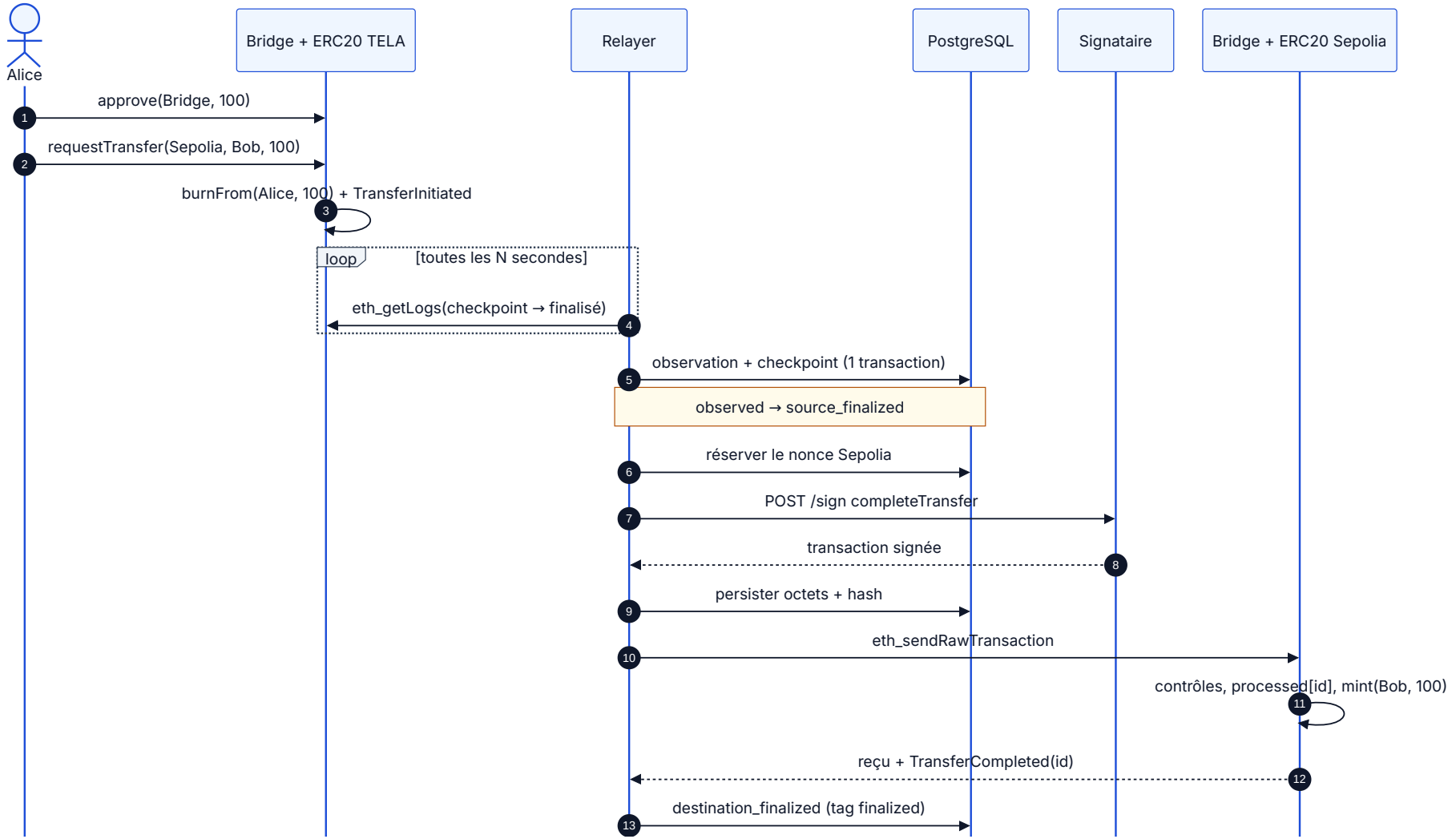


Figure 9 — Diagramme UML de séquence, cas nominal. Le sens Sepolia → TELA est symétrique, avec la finalité Sepolia (≈ 13 min) avant le mint sur TELA.

6.5 Reprise après incident : jamais deux mints

Le cas le plus délicat est la **réponse perdue** : la transaction de mint a peut-être été diffusée, peut-être pas. La règle est de ne jamais signer une seconde transaction pour le même transfert, mais de réconcilier puis, si nécessaire, de rediffuser **les mêmes octets**.

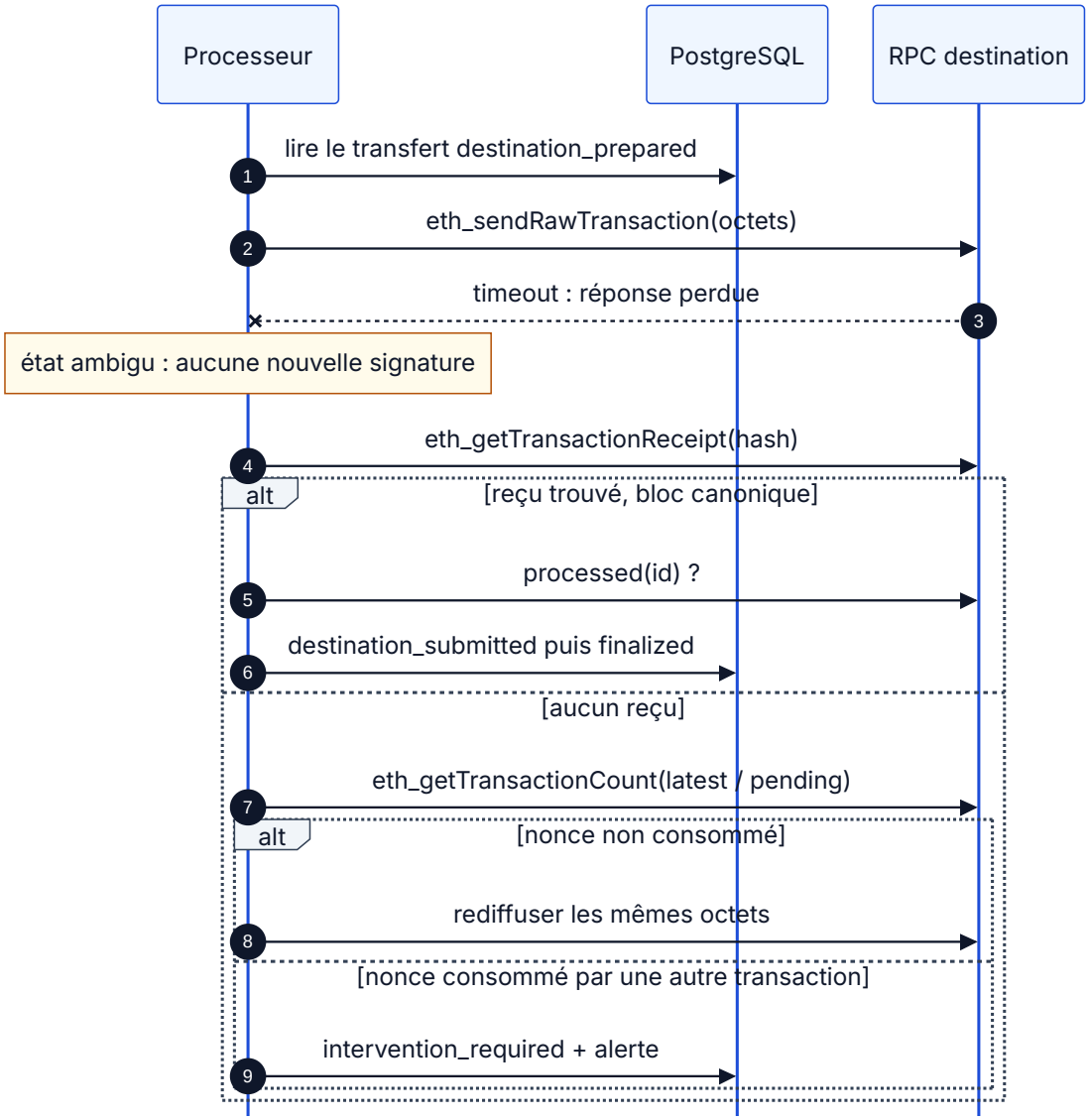


Figure 10 — Séquence de reprise. Testée sur les deux profils locaux : crash après signature persistée (sortie 91), crash après diffusion (sortie 92), RPC suspendu, concurrence de deux processus.

Paramètre de reprise (valeur par défaut)	Rôle
12 tentatives, délai exponentiel plafonné à 30 s	Borne les réessais sur erreur RPC
Timeout RPC 5 s ; 10 tentatives/minute/destination	Évite l'emballement
Transaction pending > 300 s → intervention	Nonce bloqué : décision humaine, pas de remplacement de frais automatique
Recherche d'ancêtre limitée à 128 blocs	Réorganisation plus profonde : échec explicite

6.6 Signature et gestion des clés

Le relayer ne détient **aucune clé**. Il demande la signature au signataire, qui applique une politique stricte : seule la méthode `completeTransfer` vers le Bridge configuré, valeur native nulle, montant ≤ plafond, gas borné, fréquence limitée, authentification par jeton de service.

Option de stockage	Description	Coût / contrat	Palier
Fichier de laboratoire	Clé en clair montée en lecture seule dans le seul conteneur signataire (état actuel)	Aucun	Local
Keystore chiffré	Format standard Ethereum (scrypt) ; mot de passe injecté au démarrage	Aucun	P1
Web3Signer (Consensys, Apache-2.0)	Service de signature distant, compatible Besu ; clés en keystore ou dans un coffre (HashiCorp Vault KV v2, cloud KMS)	Aucun	P2
Coffre OpenBao / Vault Talan	Coffre open source (OpenBao, MPL-2.0) ou coffre existant chez Talan	Aucun si existant	P2
HSM Talan	Si un HSM existe déjà, signature secp256k1 via PKCS#11 à vérifier	Hors budget s'il faut l'acheter	À explorer

Clé	Usage	Stockage cible	Rotation
Déployeur	Déploiement des contrats (ponctuel)	Hors ligne	Inutilisée après déploiement
Admin (multisig)	Rôles, paires, levée de pause	Signataires distincts, hors ligne	Remplacement d'un signataire
Pauser	Arrêt d'urgence	Poste d'astreinte, keystore	Révocation par l'admin
Exécuteur Sepolia / TELA	Mints (une clé par chaîne)	Signataire / Web3Signer	`grantRole` nouvelle clé puis `revokeRole`

6.7 Persistance et modèle de données

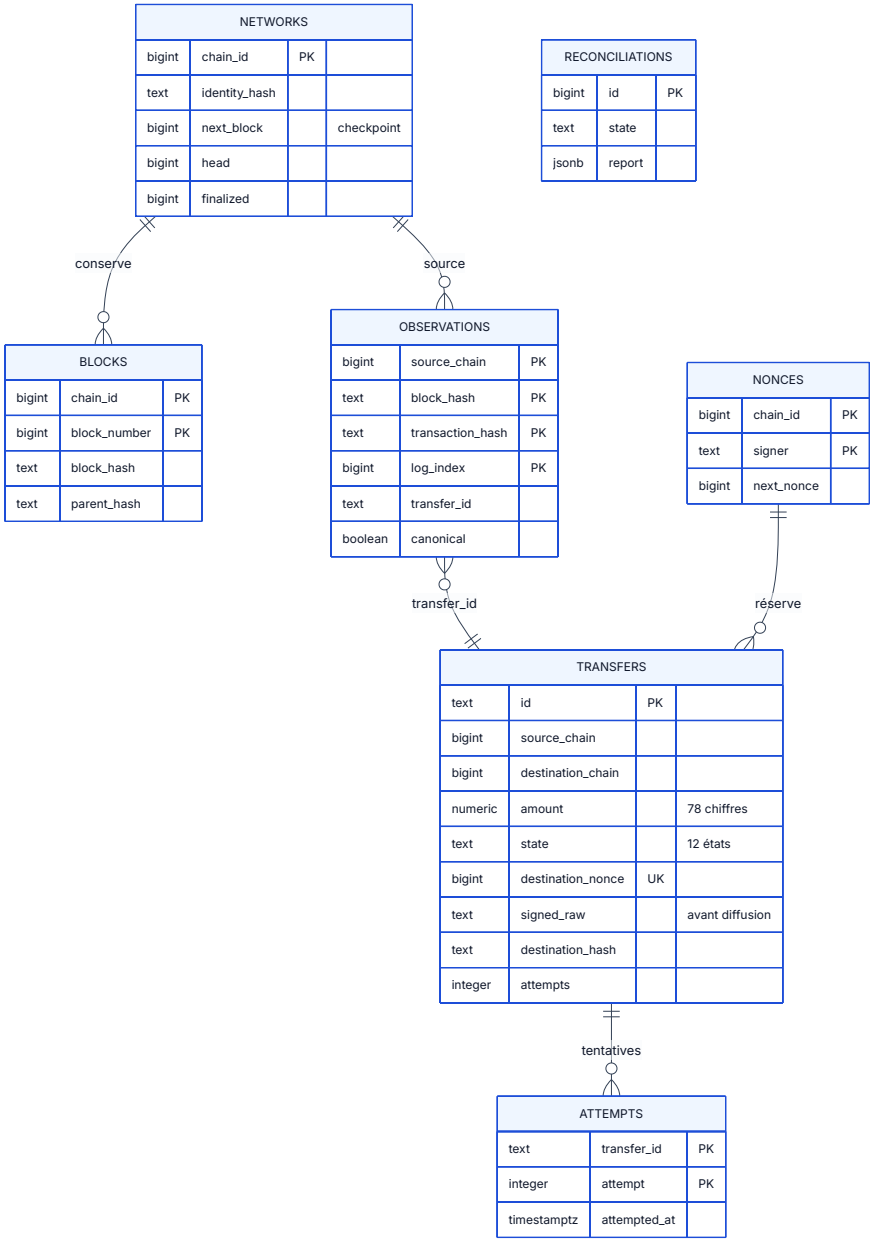


Figure 11 — Modèle de données (`src/Bridge.Relayer/schema.sql`). Unicité : identifiant de transfert, observation (bloc, tx, index de log), triplet (chaîne, signataire, nonce).

La base n'est pas la source de vérité comptable : les chaînes le sont. Une base perdue peut être reconstruite en recollectant l'historique, à condition de disposer d'un nœud qui le sert ; mais les transactions signées non diffusées seraient perdues. D'où une **sauvegarde testée** au palier 2.

6.8 Supervision, alertes et exploitation

Le relayer expose 16 métriques au format Prometheus, sans donnée personnelle ni hash en étiquette. Douze règles d'alerte existent ; le palier 1 ajoute l'Alertmanager et la notification par la messagerie interne. Principe d'honnêteté : une valeur indisponible ou périmée est affichée comme telle, jamais en vert ; les montants exacts sont contrôlés en entiers, jamais sur les flottants de Grafana.

Indicateur clé	Alerte et action
`bridge_ready`, fraîcheur de l'instantané	Relayer arrêté ou figé : redémarrer, vérifier les identités réseau
`bridge_finalized_lag_blocks`	Collecte en retard : RPC lent ou indisponible
`bridge_oldest_pending_seconds`	Transfert bloqué : runbook « transaction en attente »
`bridge_transfers{state="intervention_required"}`	Intervention humaine ; pause si nécessaire
`bridge_reconciliation_state`	`mismatch` : enquête, pause
`bridge_gas_balance_wei`	Gas de test à recharger (faucet)
`bridge_paused`	Bridge en pause : information des utilisateurs

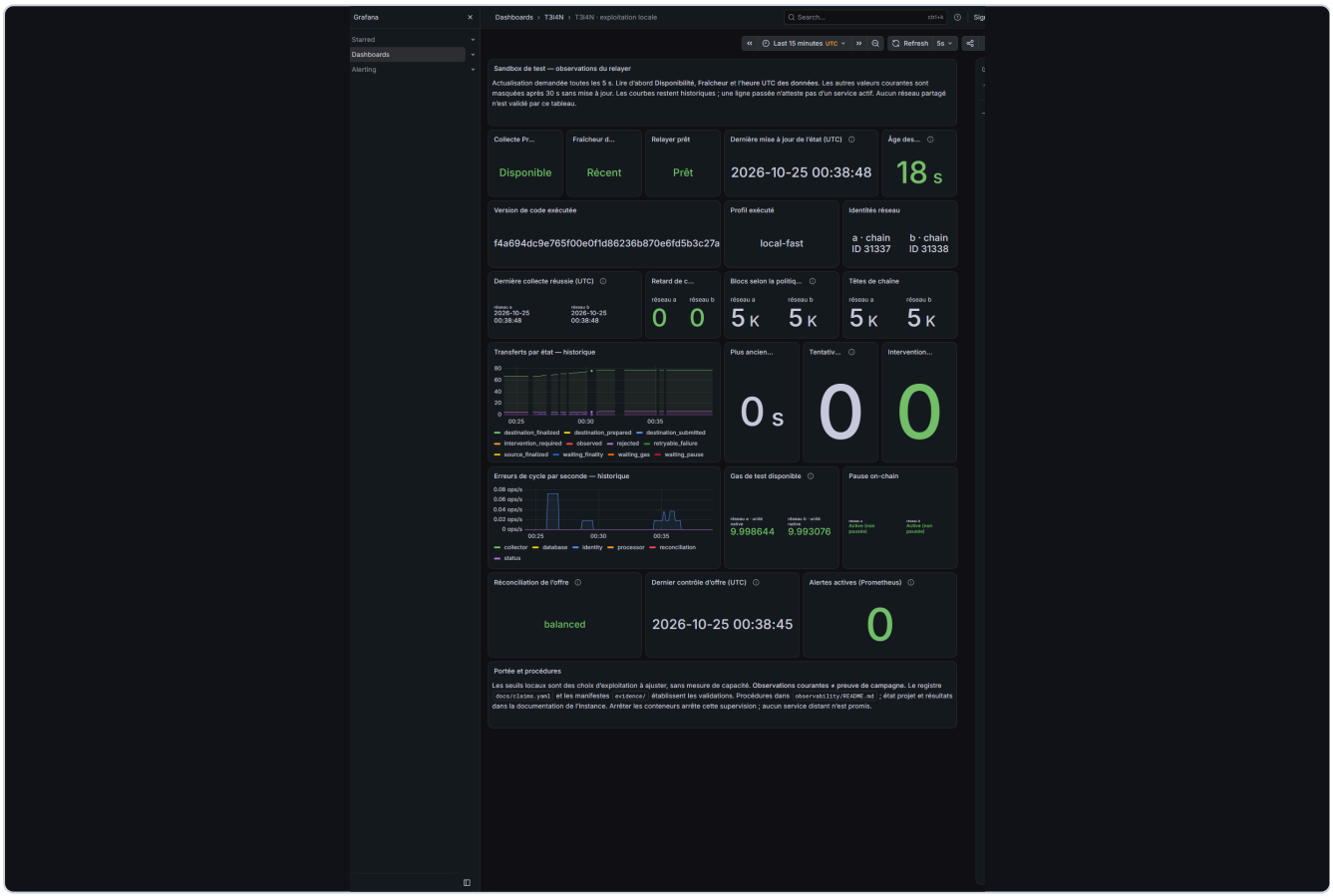


Figure 12 — Tableau d'exploitation réel (profil local-fast, 25/10/2026 00:38 UTC, observation historique) : relayer prêt, données fraîches, réconciliation `balanced`, aucune intervention.

6.9 Répartition on-chain / off-chain

Responsabilité	On-chain	Off-chain	Justification
Soldes, burn, mint	●		Seule source de vérité comptable
Anti-rejeu (`processed`)	●	○	Le contrat refuse ; la base évite de tenter
Plafonds, pause, rôles	●	○	Bornes indépendantes du relayer ; le signataire les double
Détection du burn		●	Une chaîne ne lit pas l'autre
Décision de finalité		●	Dépend du consensus de chaque réseau

Responsabilité	On-chain	Off-chain	Justification
Preuve que le burn a eu lieu	—	●	**Confiance dans le relayer** (palier 3 : signatures multiples vérifiées on-chain)
Gestion des nonces et du gas		●	Comptes externes du relayer
Réconciliation de l'offre		●	Lecture des deux chaînes à hauteur fixe

● responsabilité principale · ○ contrôle complémentaire

6.10 Alternatives étudiées

Option	Fonctionne avec TELA privée ?	Contrat / SaaS	Modèle de confiance	Verdict
Bridge maison (démonstrateur)	Oui	Aucun	Relayer autorisé + plafonds ; évolutif vers M-de-N	RETENU
Hyperlane auto-hébergé	Oui : déploiement sans permission ; validateurs et relayer à opérer soi-même	Aucun (open source)	Module de sécurité multisig configurable ; routes « synthétiques » burn/mint	PALIER 3 à étudier
Chainlink CCIP	Non : réseaux pris en charge par l'opérateur uniquement	Oui	Réseau d'oracles tiers	EXCLU
LayerZero, Axelar, Wormhole	Pas sans intégration par l'éditeur ou vérificateurs tiers	Dépendance à un tiers	Vérificateurs externes	EXCLU
Client léger / preuves ZK	Théoriquement	Aucun	Cryptographique	R&D hors portée

Pourquoi garder le bridge maison ? Il est déjà écrit, testé et compris ; il respecte les contraintes (ni contrat, ni budget) ; sa surface est petite (≈ 280 lignes Solidity). Hyperlane deviendrait pertinent si l'on voulait un modèle à validateurs multiples éprouvé plutôt que de le développer.

7 Infrastructure et réseau

7.1 Faut-il un nœud Sepolia chez nous ?

Le relayer doit lire Sepolia (événements de burn, finalité, offre) et y envoyer des transactions de mint. Quatre façons d'y accéder ont été comparées au regard des contraintes du mail.

Critère	A. RPC public sans compte	B. Nœud interne Besu + Teku	C. Client léger (Helios) + RPC public	D. Fournisseur SaaS (Infura, Alchemy…)
Contrat / budget	AUCUN conditions d'utilisation à faire valider	AUCUN	AUCUN	COMPTE + CGU exclu par le mail
Confiance	FAIBLE le RPC peut mentir ; il entre dans la frontière de confiance	MAÎTRISÉE validation locale de la chaîne	À ÉVALUER vérification cryptographique des réponses ; support de `eth_getLogs` à confirmer	TIERS
Disponibilité	Sans engagement ; quotas ; plages `eth_getLogs` limitées	Dépend de nous ; supervisable	Dépend du RPC amont	Engagement payant
Effort	≈ 1 JH	≈ 5 JH + synchronisation	R&D	≈ 1 JH
Infrastructure	Aucune	1 VM : 4 vCPU, 16 Go, SSD ≥ 1 To	Légère	Aucune
Réseau	HTTPS sortant via proxy	P2P sortant TCP/UDP 30303 et 9000, hors proxy HTTP	HTTPS sortant	HTTPS sortant
Recommandation	PALIER 1 si la cyber l'autorise, avec contrôles	CIBLE · PALIER 2	VEILLE	EXCLU

> **Le point de sécurité qui tranche**
>
> Sens Sepolia → TELA : le relayer minte sur TELA parce qu'un RPC lui a montré un burn finalisé sur Sepolia. Si ce RPC est un tiers, **ce tiers peut fabriquer un faux burn** et provoquer un mint injustifié sur TELA (dans la limite des plafonds). Au palier 1, les contrôles atténuent ce risque : identité réseau (chain ID, hash genesis), code des contrats, cohérence du nonce source, comparaison avec un second RPC indépendant, plafonds bas. À l'industrialisation, un nœud interne le supprime.

Pourquoi Besu + Teku ? Depuis la fusion d'Ethereum, un nœud complet associe un client d'exécution et un client de consensus. Besu est déjà la technologie de TELA : l'équipe capitalise sur les mêmes compétences, la même configuration et la même supervision. Teku, du même écosystème (Java), est documenté avec Besu pour Sepolia et supporte la synchronisation par point de contrôle. Le nœud ne valide pas de blocs : il se contente de suivre la chaîne.

Avant d'investir dans un nœud Sepolia

Vérifier au cadrage le statut de Sepolia (fin de vie annoncée au 30/09/2026, réseau toujours actif fin octobre 2026) et la date de bascule vers un éventuel successeur. Un nœud Sepolia synchronisé pour quelques mois seulement serait un mauvais investissement ; l'architecture permet de changer de testnet par configuration.

7.2 Schéma d'infrastructure cible et flux réseau

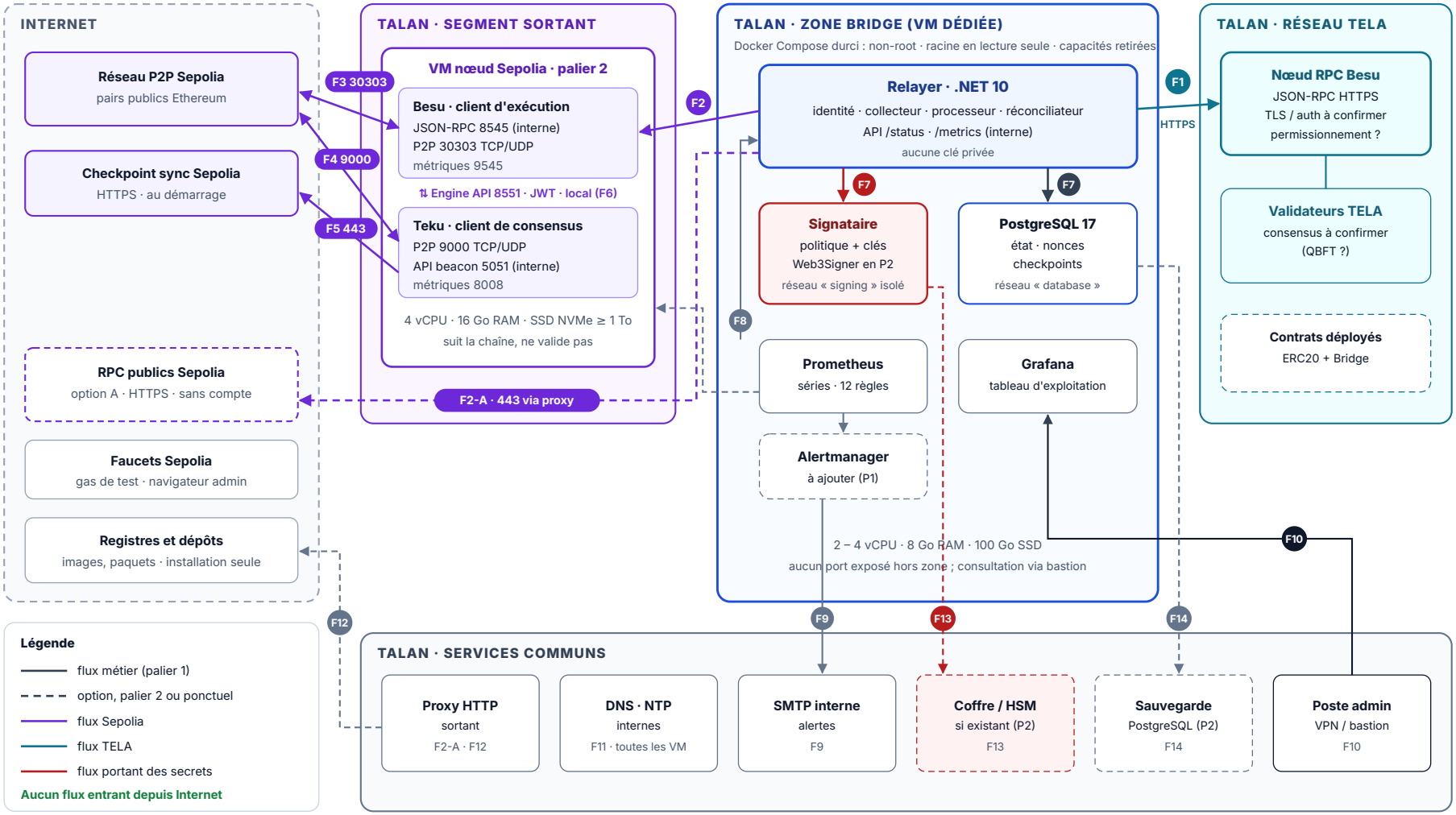


Figure 13 — Schéma d'infrastructure cible (diagramme de déploiement) et flux réseau F1 à F14. Sepolia à gauche, TELA à droite. Les ports TELA, l'emplacement exact des zones et les outils communs sont à confirmer avec Victor.

7.3 Matrice des flux réseau

#	Source	Destination	Protocole · port	Sens	Pa-lier	Justification
F1	Relayer	Nœud RPC Besu TELA	HTTPS JSON-RPC (port à confirmer)	Interne	P1	Lire logs et états TELA, envoyer les mints TELA
F2	Relayer	Besu Sepolia interne	HTTP(S) JSON-RPC 8545	Interne	P2	Lire Sepolia, envoyer les mints Sepolia
F2-A	Relayer	RPC public Sepolia	HTTPS 443 via proxy	Sortant Internet	P1	Idem F2 tant que le nœud interne n'existe pas
F3	Besu Sepolia	Pairs Sepolia	TCP + UDP 30303	Sortant (entrant facultatif)	P2	Synchronisation de la couche d'exécution
F4	Teku	Pairs Sepolia	TCP + UDP 9000	Sortant (entrant facultatif)	P2	Synchronisation de la couche de consensus
F5	Teku	Point de contrôle Sepolia	HTTPS 443	Sortant, ponctuel	P2	Démarrage rapide ; sources multiples comparées
F6	Teku	Besu Sepolia	Engine API 8551 + JWT	Local VM	P2	Couplage exécution / consensus
F7	Relayer	Signataire ; PostgreSQL	HTTP 8080 ; TCP 5432	Réseaux Compose	P1	Signature ; état persistant
F8	Prometheus	Relayer, nœuds	HTTP /metrics	Interne	P1	Supervision
F9	Alertmanager	SMTP interne	TCP 25 / 587	Interne	P1	Notification des alertes
F10	Poste admin	Grafana ; VM	HTTPS 443 ; SSH 22	Via VPN / bastion	P1	Exploitation
F11	Toutes les VM	DNS ; NTP internes	53 ; 123/UDP	Interne	P1	Résolution ; horloge fiable (horodatages, fraîcheur)
F12	VM (installation)	Registres d'images et paquets	HTTPS 443 via proxy	Sortant, ponctuel	P1	Phase d'installation distincte ; miroir interne possible
F13	Signataire	Coffre / HSM	HTTPS (TLS mutuel)	Interne	P2	Clés hors disque de la VM
F14	PostgreSQL	Sauvegarde	Selon outil Talan	Interne	P2	Restauration testée

7.4 Impact réseau

Palier 1 — impact faible

- Aucune ouverture entrante.
- Un flux HTTPS sortant (F2-A) vers un ou deux RPC Sepolia, compatible avec le proxy ; volume faible (requêtes JSON-RPC toutes les quelques secondes).
- Flux internes vers TELA (F1) et services communs.
- Installation : accès ponctuel aux registres (F12), remplaçable par un miroir.

Palier 2 — impact à instruire

- Flux pair-à-pair sortants **TCP et UDP** 30303 et 9000 vers des adresses Internet variables : non filtrables par nom, non compatibles avec un proxy HTTP.
- Ouverture entrante **facultative** (plus de pairs) : non demandée.
- Bande passante continue : prévoir au moins 25 Mbit/s et plusieurs centaines de Go par mois, à mesurer pendant la synchronisation (nombre de pairs limitable).
- Synchronisation initiale : de quelques heures à quelques jours, gros volume ponctuel.

Points de vigilance réseau : horloge (NTP) fiable pour les horodatages et la fraîcheur des métriques ; résolution DNS des RPC ; TLS d'interception du proxy (certificat Talan à injecter dans les clients) ; pas de WebSocket requis ; limitation des réponses `eth_getLogs` par les RPC publics (pagination déjà implémentée par lots de 32 à 512 blocs).

7.5 Dimensionnement

VM	vCPU	RAM	Disque	Croissance et commentaire	Palier
Bridge (relayer, signataire, PostgreSQL, supervision)	2 – 4	8 Go	100 Go SSD	Faible : quelques Mo par millier de transferts ; rétention Prometheus 15 jours	P1
Nœud Sepolia (Besu + Teku)	4	16 Go	≥ 1 To NVMe	Continue ; taille exacte à mesurer à la synchronisation (ordre de grandeur : quelques centaines de Go) ; alerte à 80 %	P2
TELA	—	—	—	Existant : aucun nœud supplémentaire, sous réserve que le RPC TELA serve l'historique des logs	—

Ces valeurs sont des points de départ prudents, non des mesures : le démonstrateur a montré qu'une ressource sous-dimensionnée (mémoire Anvil) se traduit par un arrêt silencieux sans supervision adaptée (incident du 27/10, ADR-0006).

7.6 Environnements

Environnement	Réseaux	Usage	Statut
Local `local-fast`	Deux Anvil (31337 / 31338)	Développement, protocole, pannes	IMPLÉMENTÉ
Local `local-besu`	Besu QBFT local + Anvil	Compatibilité client Besu	IMPLÉMENTÉ
Intégration `integration-testnet`	Sepolia (11155111) + TELA de test	Premier pont réel, recette	BLOQUÉ accès
Production	—	Hors périmètre : aucun mainnet ni actif réel	HORS PÉRIMÈTRE

7.7 Exigences cyber à valider avec Victor

Domaine	Mesure proposée	État
Exposition	Aucun service exposé à Internet ; consultations via bastion/VPN ; ports liés à l'interface interne	IMPLÉMENTÉ (local)
Conteneurs	Non-root, racine en lecture seule, capacités retirées, `no-new-privileges`, ressources bornées, pas de socket Docker monté	VÉRIFIÉ LOCAL
Secrets	Aucun secret dans Git (scan Gitleaks) ; clés uniquement dans le signataire ; mots de passe en fichiers montés ; coffre en P2	VÉRIFIÉ LOCAL
Chaîne d'approvisionnement	Images épinglées par empreinte, dépendances verrouillées (NuGet, Python), OpenZeppelin vendoré avec SHA-256	IMPLÉMENTÉ
Journalisation	Journaux JSON sans secret ni payload signé ; export vers le SIEM Talan si requis	PROPOSÉ
Flux internes	TLS entre relayer, signataire et base (aujourd'hui en clair sur réseaux internes du laboratoire)	PROPOSÉ P2
Gouvernance des rôles	Admin multisig, séparation pauser / exécuteur / admin, procédure de rotation	PROPOSÉ P2
Hôte	VM dédiée au projet, durcie selon le standard Talan ; l'administrateur Docker reste une autorité étendue	BLOQUÉ VM

8 Composants logiciels requis

8.1 Inventaire

#	Composant	Responsabilité	Technologie · licence	État · palier
C1	**Token ERC20** (×2)	Soldes ; `mint` et `burnFrom` réservés au Bridge	Solidity, OpenZeppelin 5.4 · MIT	EXISTANT à adapter au token Talan (P1)
C2	**Contrat Bridge** (×2)	Burn + événement ; mint anti-rejeu ; plafonds ; pause ; rôles	Solidity 0.8.30, Foundry · MIT	VÉRIFIÉ LOCAL
C3	**Relayer**	Identité, collecte, finalité, nonces, soumission, reprise, réconciliation, API	.NET 10 LTS, Nethereum 6.1 · MIT	VÉRIFIÉ LOCAL profil testnet à activer (P1)
C4	**Signataire**	Seul détenteur des clés ; politique de signature	.NET 10 ; Web3Signer (Apache-2.0) en P2	VÉRIFIÉ LOCAL durcissement P2
C5	**Base d'état**	Observations, transferts, nonces, checkpoints, tentatives, réconciliations	PostgreSQL 17 · licence PostgreSQL	VÉRIFIÉ LOCAL sauvegarde P2
C6	**Accès TELA**	RPC Besu TELA existant	Hyperledger Besu · Apache-2.0	BLOQUÉ accès (P1)
C7	**Accès Sepolia**	RPC public contrôlé (P1) puis nœud Besu + Teku (P2)	Besu, Teku · Apache-2.0	PROPOSÉ
C8	**Supervision**	Métriques, tableaux, règles, notification	Prometheus, Alertmanager · Apache-2.0 ; Grafana · AGPL-3.0	VÉRIFIÉ LOCAL Alertmanager P1
C9	**Outillage et preuves**	Commandes `bootstrap/up/test/prove`, manifestes, CI	Python 3.12, Docker Compose	IMPLÉMENTÉ
C10	**IHM de suivi** (option)	Bridger depuis un navigateur, suivre un identifiant	Page statique + bibliothèque web3 (MIT)	OPTION 3 – 6 JH
C11	**Multisig d'administration**	Admin des contrats à plusieurs signatures	Safe (contrats open source)	PROPOSÉ P2

Grafana est sous AGPL-3.0 : usage interne non modifié sans contrainte particulière ; à signaler à la cyber si une politique de licences existe. Aucune licence commerciale n'est requise.

8.2 Interfaces

Interface	Contrat	Sécurité
`requestTransfer(dest, recipient, amount)`	Appel utilisateur après `approve` ; retourne l'identifiant ; émet `TransferInitiated`	Pause, plafond, paire
`completeTransfer(src, srcBridge, srcToken, nonce, amount, recipient)`	Appel de l'exécuteur ; émet `TransferCompleted`	Rôle, paire, plafonds, anti-rejeu

Interface	Contrat	Sécurité
`POST /sign` (signataire)	JSON : `chainId`, `nonce`, `gasPriceWei`, `gasLimit`, `to`, `data` → transaction brute, hash, adresse	Jeton de service, corps ≤ 4 096 octets, politique ; 401 / 400 / 403 / 429
`GET /status`, `/health` (relayer)	État prêt/périmé, réseaux, comptes par état, réconciliation	Réseau interne
`GET /metrics` (relayer)	16 métriques Prometheus	Réseau interne
`GET /transfers/{id}`	État d'un transfert pour l'utilisateur ou l'IHM	**PROPOSÉ** P1
Configuration `BRIDGE_CONFIG`	JSON : réseaux, chain ID, hash genesis, adresses, empreintes de code, finalité, plafonds, gas	Aucun secret ; secrets en fichiers séparés

8.3 Pile technique et versions

Élément	Version épinglée	Pourquoi
Solidity / cible EVM	0.8.30 · London	Compatible avec Besu sans supposer PUSH0/Cancun ; à confirmer selon les forks TELA
OpenZeppelin Contracts	5.4.0 (vendoré, SHA-256)	Rôles, ERC20, pause, réentrance éprouvés
Foundry (forge) / Anvil	1.3.1 / 1.8.4	Tests, fuzzing, invariants ; nœuds locaux
.NET SDK / runtime	10.0.402 / 10.0.13 (LTS)	Support long ; compétences Talan
Nethereum · Npgsql	6.1.0 · 10.0.3	Client Ethereum .NET ; pilote PostgreSQL
Besu (local)	26.10.0	Même client que TELA ; version TELA à confirmer
PostgreSQL · Prometheus · Grafana	17.11 · 3.13.1 · 13.2.4	Verrous et transactions ; supervision

Les images sont épinglées par empreinte dans `sandbox/versions.lock.json` ; toute montée de version invalide les preuves précédentes et impose une nouvelle campagne.

9 Démonstration

Deux niveaux de démonstration : une exécution **rejouée le jour de ce document**, centrée sur le protocole on-chain, et les **campagnes complètes** du démonstrateur (relayer, base, signataire, supervision, pannes), qui sont historiques.

9.1 Démonstration rejouée le 30 octobre 2026

29 / 29
contrôles réussis, code de sortie 0

13 s
13:39:58 → 13:40:11 UTC, compilation et tests inclus

18
transactions réelles sur deux chaînes locales

Commande	`python3 livrables/livrable-1/demo/demo_burn_mint.py --tools <Foundry> --solc <solc 0.8.30> --out evidence/demo-livrable1-20261030T133958Z`
Commit testé	`2f05504386b08a31be2270fed378a237c7185393`, arbre propre ; le rapport est ajouté dans un commit ultérieur
Outils	forge 1.3.1 (tests), anvil et cast 1.8.4 (commit `50af4efe`, identique au build retenu par l'ADR-0006), solc 0.8.30 vérifié par SHA-256 contre `contracts/solc.lock.json`
Réseaux	Deux Anvil locaux, chaîne A = 31337, chaîne B = 31338, hardfork London, liés à 127.0.0.1 ; comptes Anvil publics, jamais utilisables sur un réseau partagé
Rôle du relayer	Joué pas à pas par le script : lecture du log, 3 confirmations locales, recalcul indépendant de l'identifiant, soumission par le compte exécuteur
Preuves	`evidence/demo-livrable1-20261030T133958Z/` : `demo.json`, `forge-test.log`, `console.log`, `index.json`, `SHA256SUMS`

> **Portée**
> Cette démonstration valide le protocole on-chain et la logique du relayer de façon manuelle. Elle n'exerce ni le relayer .NET, ni PostgreSQL, ni le signataire, ni la supervision, et **ne valide ni Sepolia ni TELA**. Les outils proviennent des archives officielles et non des images Docker épinglées (moteur Docker indisponible dans l'environnement d'exécution).

Scénarios et résultats

Id	Scénario	Attendu et observé	Résultat
T	27 tests Solidity (unitaires, fuzzing, invariants)	27 réussis, 0 échec, 0 ignoré	RÉUSSI
S1	Aller A → B de 100 tEUR (Alice → Bob)	Burn, transit = 100, 3 blocs, ID recalculé identique, mint, `processed` = true, solde Bob = 100	RÉUSSI
S2	Rejeu du même transfert	Simulation refusée (`TransferAlreadyProcessed`) ; transaction forcée incluse avec statut 0 ; aucun second mint	RÉUSSI
S3	Mint par un compte non autorisé	`AccessControlUnauthorizedAccount`	RÉUSSI
S4	Transfert de 1 001 tEUR (plafond 1 000)	`InvalidAmount`	RÉUSSI
S5	Pause d'urgence	Burn refusé (`EnforcedPause`) ; le pauser ne peut pas lever la pause ; l'admin la lève	RÉUSSI
S6	Retour B → A de 40 tEUR (Bob → Alice)	Burn, 3 blocs, ID identique, mint, offre B = 60	RÉUSSI

Id	Scénario	Attendu et observé	Résultat
S7	Réconciliation au repos	Offre A + offre B + transit = 1 000 000 tEUR	RÉUSSI
S8	**Limite de confiance** : mint de 5 tEUR fabriqué par l'exécuteur, sans burn	Le contrat **accepte** (attendu, c'est la limite du modèle) ; la réconciliation mesure un écart de +5 tEUR	RÉUSSI

Évolution de l'offre

Étape	Offre A	Offre B	Transit	Total	Réconciliation
Initial (1 000 000 tEUR mintés à Alice sur A)	1 000 000	0	0	1 000 000	balanced
S1 — burn de 100 sur A	999 900	0	100	1 000 000	balanced
S1 — mint de 100 sur B	999 900	100	0	1 000 000	balanced
S2 — rejeu refusé	999 900	100	0	1 000 000	balanced
S6 — burn de 40 sur B	999 900	60	40	1 000 000	balanced
S6 — mint de 40 sur A	999 940	60	0	1 000 000	balanced
S8 — mint fabriqué de 5 sur B	999 940	65	0	1 000 005	mismatch +5

Transactions exécutées

Action	Ch.	Compte	Hash	Bloc	Gas	Statut
Déploiement jeton A	A	admin	`0xa5a65bf1c565562e…2f96b6`	—	846 720	1 · succès
Déploiement jeton B	B	admin	`0x91cf1d38e1343315…9cd060`	—	806 836	1 · succès
Déploiement bridge A	A	admin	`0x39108b75092652e3…4bb2dd`	—	1 250 368	1 · succès
BRIDGE_ROLE accordé au bridge A	A	admin	`0x8924b57990c23eee…69f8e3`	3	51 520	1 · succès
Déploiement bridge B	B	admin	`0xb23cc1e151e22fa4…a61cae`	—	1 250 368	1 · succès
BRIDGE_ROLE accordé au bridge B	B	admin	`0xf31a48b22b53d30b…1e8f7f`	3	51 520	1 · succès
Paire A → B (définitive)	A	admin	`0x04ef9119c1e5be64…05e6b5`	4	71 616	1 · succès
Paire B → A (définitive)	B	admin	`0xe51cb13f4b0fe134…ea1596`	4	71 616	1 · succès
S1 : approve Alice → bridge A	A	alice	`0x849831ec809a87fc…6f9b5c`	5	46 413	1 · succès
S1 : requestTransfer (burn)	A	alice	`0xf3be5d22c3ed969d…c67d01`	6	76 691	1 · succès
S1 : completeTransfer (mint)	B	exécuteur	`0x91afae79bfe30529…492397`	5	134 296	1 · succès
S2 : rejeu forcé (gas fixé, sans estimation)	B	exécuteur	`0x401a551a861b6ffe…251c14`	6	40 471	0 · annulée
S5 : pause	A	pauser	`0x103dd039bfdf2148…e0bd0e`	10	47 066	1 · succès
S5 : levée de pause	A	admin	`0x8f1dc248a3cf5ae4…750a6d`	11	25 125	1 · succès
S6 : approve Bob → bridge B	B	bob	`0xeda2691b2a77ff53…7b433b`	7	46 413	1 · succès
S6 : requestTransfer (burn)	B	bob	`0x245459cee77e47eb…d7f1df`	8	76 691	1 · succès
S6 : completeTransfer (mint)	A	exécuteur	`0x3827b572c9f4908c…4c31dd`	12	100 096	1 · succès
S8 : completeTransfer sans burn (fraude simulée)	B	exécuteur	`0x4352e90d99e66b7e…b28e04`	12	100 096	1 · succès

Identifiants de transfert : S1 `0x4192a37889d59ad3c96c0da2282e207ea3d8a5b53dfcbd3dc9f6a0b280bae643` ; S6 `0x9a8fae30fa6b6b14f44b1fab4a16a2fe5de7af29bb24ebbc2aed504cc3d5e2a6`. Les deux chaînes ayant le même déployeur, les contrats ont les mêmes adresses sur A et B : seule l'identité de chaîne les distingue, d'où son contrôle systématique.

> **Ce que la démonstration apprend pour Sepolia**
>
> Gas mesuré : burn ≈ 77 k, approve ≈ 46 k, mint ≈ 100 à 134 k (le premier mint d'une époque écrit un nouvel emplacement). Un transfert coûte donc ≈ 0,22 à 0,26 M de gas (approve + burn + mint), dont ≈ 0,10 à 0,13 M payé par le relayer pour le mint. À 1 à 10 gwei, un mint coûte entre 0,0001 et 0,0013 SepoliaETH : quelques dixièmes de SepoliaETH suffisent pour des centaines de transferts de test.

9.2 Campagnes complètes du démonstrateur (25 octobre 2026)

La commande `prove` exécute, sur un profil, l'ensemble des contrôles et scelle un manifeste (commandes, codes de sortie, versions, images, empreintes). Les deux campagnes ci-dessous portent sur le commit `b97c541` et sont **HISTORIQUES** : le code a changé depuis (§3.2).

Contrôle	Attendu	local-fast	local-besu
tooling	Tests d'outillage Python sans skip	✔	✔
capabilities	50 agents et 50 skills valides	✔	✔
compose	Configurations durcies, sans port public	✔	✔
docs	Documentation MkDocs stricte	✔	✔
secrets	Gitleaks : aucun secret détecté	✔	✔
contracts	Compilation London, tests, fuzz et invariants	✔	✔
dotnet	64 tests signataire, 51 tests relayer, sans réseau	✔	✔
anvil_history	520 blocs, historique conservé après deux arrêts	✔	✔
startup	Identités vérifiées, contrats appariés, relayer prêt	✔	✔
hardening	Restrictions effectivement appliquées	✔	✔
isolation	Accès permis réussis, accès interdits refusés	✔	✔
application_isolation	Clés au seul signataire, signature non authentifiée refusée	✔	✔
roundtrip	Aller-retour réel de 10 tokens, offre au repos conservée	✔	✔
resilience	Scénarios de panne (9 sur Anvil, 7 applicables sur Besu)	✔	✔
persistence	Données et progression conservées après arrêt/relance	✔	✔
monitoring	Règles, requêtes et tableau sur métriques réelles	✔	✔
integration_gate	Intégration réelle refusée avant toute commande	✔	✔
source_unchanged	Sources inchangées pendant la campagne	✔	✔
Total — 2026-10-25, 00:24:52 → 00:32:48 UTC et 00:33:58 → 00:40:29 UTC		**18 / 18**	**18 / 18**

Non exécutés et non validés : intégration Sepolia, intégration TELA ; sur Besu, gas forcé à zéro et réorganisation (scénarios propres à Anvil). La CI GitHub Actions du même commit est également réussie (18 contrôles local-fast sur clone neuf).

9.3 Scénario de démonstration proposé à Alain (20 minutes)

Durée	Séquence	Ce qu'on montre
3 min	Schéma de contexte et principe burn & mint (figures 1 et 3)	Qui fait quoi, où est la confiance

Durée	Séquence	Ce qu'on montre
3 min	`python3 livrables/livrable-1/demo/demo_burn_mint.py …`	29 contrôles en direct, dont le rejeu refusé et la fraude détectée
6 min	`python scripts/bridge.py up --profile local-besu` puis `demo` (préparé avec `bootstrap`)	Relayer réel, PostgreSQL, Besu QBFT ; état sur `/status`
4 min	Tableau Grafana pendant un transfert, puis arrêt du relayer	Supervision honnête : donnée périmée signalée, reprise sans double mint
4 min	Limites et prochaines étapes (§13)	Décisions attendues

Prérequis : poste ou VM Linux avec Docker, images préparées la veille (`bootstrap` utilise le réseau), environ 8 Go de RAM libres pour `local-besu`.

10 Analyse des risques projet

10.1 Méthode

Chaque risque est coté en **probabilité** (1 rare, 2 possible, 3 probable, 4 quasi certain) et en **impact** (1 mineur, 2 retard limité, 3 retard important ou incident de test, 4 échec du projet ou incident de sécurité). La criticité est le produit : ≥ 12 critique, 8 – 9 élevée, 4 – 6 modérée, ≤ 3 faible. Les cotations reflètent l'état au 30/10/2026 et seront revues à chaque jalon.

10.2 Cartographie

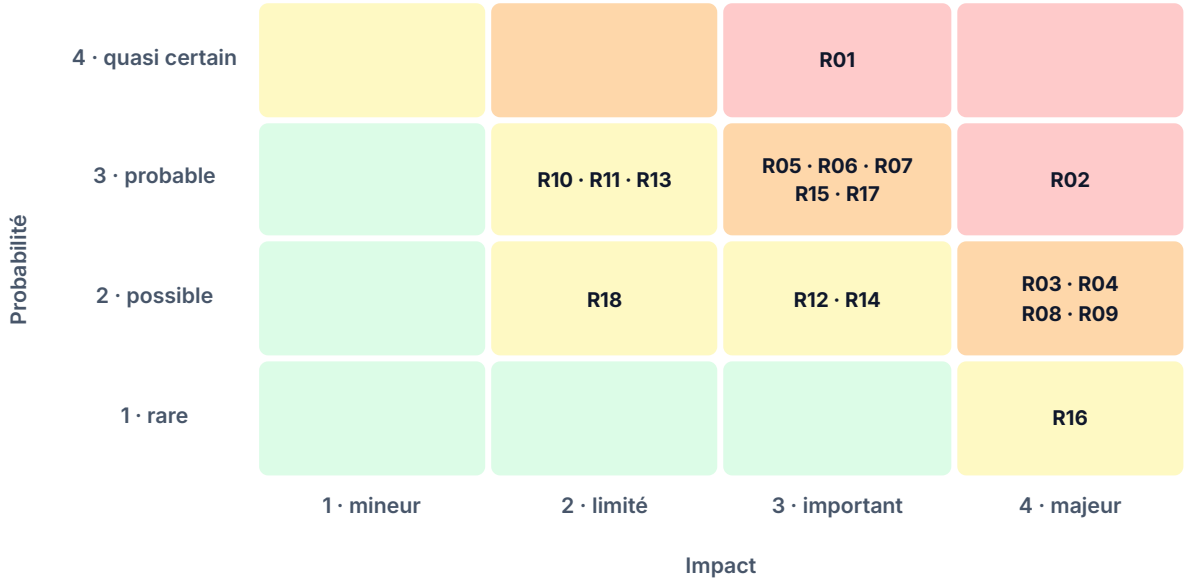


Figure 14 — Matrice probabilité × impact des 18 risques identifiés.

10.3 Registre des risques

Id	Risque	Catégo-rie	P	I	Crit.	Mesures de réduction	Porteur
R01	Disponibilité insuffisante du porteur (sujet non prioritaire, alternance, examens)	Projet	4	3	12	Paliers courts et autonomes ; tâches de 1 à 3 jours ; documentation continue ; automatisation ; calen-drier fondé sur une disponibilité déclarée (D5)	Alain, porteur
R02	Flux réseau refusés ou très re-tardés (P2P Sepolia, RPC sortant)	Infra	3	4	12	Dossier de flux au cadrage ; option A sans P2P ; aucun flux entrant ; alternatives B/C	Victor, cyber
R03	Compromission de la clé exé-cuteur ou de l'accès au signa-taire : mints sans burn	Sécurité	2	4	8	Clés hors relayer ; politique du signataire ; pla-fonds bas ; pause ; réconciliation et alerte ; Web3Signer + coffre (P2) ; attestations multiples (P3)	Porteur, cyber
R04	RPC Sepolia tiers mensonger ou incohérent (palier 1)	Sécurité	2	4	8	Contrôle d'identité ; cohérence du nonce source ; double source comparée ; plafonds ; nœud interne au palier 2	Porteur
R05	Pérennité de Sepolia (fin de vie annoncée au 30/09/2026, successeur à confirmer)	Projet	3	3	9	Vérifier au cadrage ; architecture indépendante du testnet ; pas de nœud Sepolia avant décision ; mi-gration ≈ 2 – 4 JH	Alain, Victor

Id	Risque	Catégorie	P	I	Crit.	Mesures de réduction	Porteur
R06	Token « Stablecoin Euro Talan » incompatible avec le burn/mint	Archi	3	3	9	Analyse dès le cadrage ; arbre de décision §5.5 ; options 2 ou 3 chiffrées	Porteur, Victor
R07	Accès TELA tardif ou incomplet (RPC, comptes, droits de déploiement)	Projet	3	3	9	Questionnaire §13.2 ; demandes groupées ; travail local en parallèle ; profil Besu local comme substitut technique	Victor
R08	Finalité mal paramétrée (réorganisation Sepolia, consensus TELA mal compris)	Archi	2	4	8	Tag `finalized` uniquement ; aucun repli silencieux ; confirmation écrite du consensus TELA ; tests de réorganisation	Porteur, Victor
R09	Défaut dans les contrats (logique, rôles, cas limites)	Sécurité	2	4	8	27 tests dont fuzzing et invariants ; code court sans proxy ; relecture indépendante (P2) ; plafonds immuables	Porteur, relecteur
R10	Transaction bloquée : nonce en attente, pic de frais Sepolia	Exploitation	3	2	6	Plafond de frais et attente ; alerte à 300 s ; runbook ; remplacement de frais assisté (évolution)	Porteur
R11	Pénurie de SepoliaETH (faucets limités, pas de budget)	Exploitation	3	2	6	Faucet à preuve de travail sans compte ; réserve constituée à l'avance ; alerte de solde ; faible consommation (≈ 0,13 M gas par mint)	Porteur
R12	Perte ou corruption de la base PostgreSQL	Exploitation	2	3	6	Sauvegarde et restauration testées (P2) ; recollecte depuis les chaînes ; contraintes d'unicité	Porteur, infra
R13	Saturation ou désynchronisation du nœud Sepolia (disque, mémoire)	Infra	3	2	6	Supervision disque et retard ; alerte à 80 % ; repli en lecture sur RPC public contrôlé	Infra
R14	Émission hors bridge par un administrateur du token	Gouvernance	2	3	6	Admin multisig ; inventaire des rôles ; réconciliation ; journal des changements de rôles	Alain, cyber
R15	Dérive de périmètre vers une « production » prématurée	Projet	3	3	9	Paliers et go/no-go ; critères d'acceptation écrits ; mention « test » sur tous les artefacts	Alain
R16	Confusion entre test et réel ; enjeu réglementaire d'un stablecoin euro	Conformité	1	4	4	Testnets uniquement, tokens sans valeur ; avis juridique avant tout usage réel	Alain
R17	Dépendance à une seule personne (porteur) et à un seul référent TELA	Projet	3	3	9	Dépôt documenté ; ADR ; runbooks ; démonstration rejouable en une commande ; binôme pour la revue	Alain
R18	Vulnérabilités ou obsolescence des dépendances (Nethereum, images, évolutions EVM)	Sécurité	2	2	4	Versions épinglées ; audit NuGet ; scans ; cible London ; mise à jour planifiée avec nouvelle campagne	Porteur

10.4 Modèle de menaces (synthèse)

Compromission ou défaut	Conséquence possible	Protection existante	Renforcement proposé
Clé d'exécution	Mints frauduleux dans les plafonds	Plafonds immuables, pause, réconciliation	Coffre, rotation, M-de-N
Relayer ou jeton du signataire	Demandes frauduleuses conformes à la politique	Politique stricte, fréquence limitée	Signataire qui vérifie le burn sur son propre nœud
RPC malveillant	Faux burn, faux reçus	Contrôles d'identité et de cohérence	Nœud interne, double source
Administrateur du Bridge	Rôles réattribués, pause levée	Paire immuable, rôles séparés	Multisig, journal d'administration

Compromission ou défaut	Conséquence possible	Protection existante	Renforcement proposé
Administrateur du token	Émission hors bridge	Réconciliation (partielle)	Multisig, inventaire des rôles
Base de données	Perte de l'état de reprise, falsification	Contraintes, compte applicatif non admin	Sauvegarde, TLS, contrôle d'accès
Hôte ou moteur Docker	Accès aux clés et volumes	Conteneurs durcis (insuffisant seul)	VM dédiée, coffre, durcissement Talan
Finalité invalidée	Mint d'un burn annulé	Finalité par réseau, détection de réorganisation	Tag `finalized`, intervention documentée

11 Estimation des charges

11.1 Hypothèses de chiffrage

- **Unité** : jour-homme (JH) de 7 heures de travail effectif du porteur.
- **Méthode** : estimation à trois points (optimiste, probable, pessimiste) par lot, combinée par la formule PERT `(O + 4P + Pe) / 6`, puis 10 % de pilotage et d'imprévus.
- **Inclus** : conception, code, tests, preuves, documentation et démonstration de chaque lot.
- **Exclu** : délais d'attente externes (accès, validations), temps des autres équipes (chiffré à part), palier 3.
- **Point de départ** : le démonstrateur existant (§3). Sans lui, la charge serait nettement supérieure.
- **Incertitude** : forte tant que TELA, le token existant et les règles réseau ne sont pas connus. La réalisation du démonstrateur ne constitue pas une mesure de productivité transposable. Le chiffrage est réévalué à la fin du cadrage (L0).

11.2 Charges par lot

Palier 1 — premier pont réel Sepolia ⇄ TELA de test

Lot	Intitulé	Contenu	Opt.	Prob.	Pess.	PERT
L0	Cadrage et accès	Rendez-vous Victor, questionnaire, accès TELA de test, choix Sepolia/successeur, dossier de flux, analyse du token existant	2	3	5	3,2
L1	Socle VM dédiée	VM durcie, Docker Compose, déploiement du démonstrateur, nouvelle campagne `prove` sur les deux profils	2	3	5	3,2
L2	Accès Sepolia contrôlé	RPC public validé, identité réseau, double source, limites de plage, proxy et certificat	1	1,5	3	1,7
L3	Contrats et token	Adaptation au token « Stablecoin Euro Talan » (option 1 ou 2), tests, déploiement sur les deux testnets	2	4	8	4,3
L4	Relayer testnets	Profil `integration-testnet`, finalité `finalized` et TELA, TLS/authentification RPC, transactions EIP-1559, route `/transfers/{id}`	4	6	10	6,3
L5	Clés et gas de test	Keystores chiffrés, comptes séparés, provision de SepoliaETH, Alertmanager + SMTP	1	2	3	2,0
L6	Recette nominale	Allers-retours réels, réconciliation, preuves scellées, rapport	2	3	5	3,2
L7	Documentation et démo	Mise à jour de ce document, démonstration à Alain et Victor	1	1,5	2	1,5
Sous-total			**15**	**24**	**41**	**25,3**
Pilotage et imprévus (10 %)						**2,5**
Total palier 1			**17**		**45**	**≈ 28**

Palier 2 — industrialisation

Lot	Intitulé	Contenu	Opt.	Prob.	Pess.	PERT
M1	Nœud Sepolia interne	Besu + Teku, synchronisation, supervision, bascule du relayer, procédure de resynchronisation	3	5	8	5,2
M2	Signature durcie	Web3Signer + coffre (OpenBao ou existant Talan), rotation des clés, admin multisig	3	5	8	5,2
M3	Exploitation	Sauvegarde/restauration testées, runbooks (pause, nonce bloqué, réorganisation, rotation), astreinte « au mieux »	3	5	8	5,2

Lot	Intitulé	Contenu	Opt.	Prob.	Pess.	PERT
M4	Pannes et campagne testnets	Injections de panne sur testnets, campagne de preuves scellée, tableau de bord projet	3	5	8	5,2
M5	Sécurité	Modèle de menaces, durcissement, TLS interne, corrections issues de la relecture	2	4	6	4,0
M6	Passation	Documentation d'exploitation, transfert à l'équipe qui opérera le bridge	1	2	3	2,0
Sous-total			**15**	**26**	**41**	**26,7**
Pilotage et imprévus (10 %)						**2,7**
Total palier 2			**17**		**45**	**≈ 29**

≈ 57 JH
paliers 1 + 2 (fourchette 33 – 90 JH), porteur seul

≈ 26 JH
palier 3 optionnel « confiance renforcée » (attestations multiples ou Hyperlane), à rechiffrer après étude

11.3 Charges des autres contributeurs

Contributeur	Contribution attendue	Palier 1	Palier 2
Victor (référent TELA)	Présentation de TELA, accès, revue d'architecture, déploiements TELA	2 – 3 JH	1 – 2 JH
Infrastructure / réseau Talan	VM, flux, proxy, DNS/NTP, SMTP, sauvegarde	1 – 2 JH	1 – 3 JH
Cybersécurité Talan	Validation des flux et de l'architecture, secrets, revue	1 – 2 JH	1 – 3 JH
Relecteur indépendant	Relecture du code Solidity et .NET, des tests et des preuves	—	3 – 5 JH
Alain	Arbitrages, jalons, go/no-go	0,5 – 1 JH	0,5 – 1 JH
Total autres contributeurs		**4,5 – 8 JH**	**6,5 – 14 JH**

11.4 Planning indicatif et scénarios de disponibilité

Le sujet passe après les missions de Talan et du client : la durée calendaire dépend d'abord de la disponibilité réelle du porteur.

Disponibilité moyenne du porteur	Palier 1 (≈ 28 JH)	Palier 2 (≈ 29 JH)	Paliers 1 + 2
½ jour par semaine (≈ 3 à 4 h)	≈ 56 semaines · 13 mois	≈ 58 semaines	> 2 ans
1 jour par semaine (hypothèse retenue)	**≈ 28 semaines · 6,5 mois**	≈ 29 semaines	≈ 13 mois
2 jours par semaine	≈ 14 semaines · 3,5 mois	≈ 15 semaines	≈ 7 mois

Hors congés, périodes d'examens et délais externes. Les attentes d'accès se recouvrent avec le travail local mais peuvent bloquer L3 et L4.

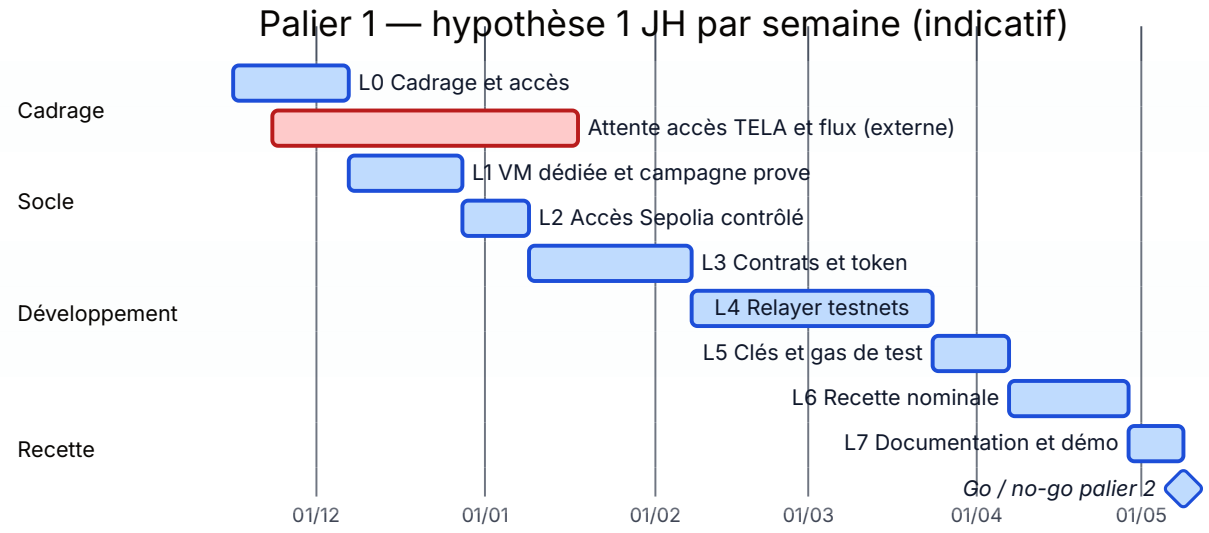


Figure 15 — Planning indicatif du palier 1 à partir du 16/11/2026 : fin vers mi-mai 2027 à 1 JH par semaine, avant pilotage et congés.

11.5 « Zéro budget » ne veut pas dire « zéro coût »

Poste	Nature	Ordre de grandeur	Porté par
Temps du porteur	Paliers 1 et 2	≈ 57 JH	Accord d'Alain
Temps des autres équipes	Victor, infra, cyber, relecteur, Alain	≈ 11 – 22 JH	Équipes Talan
VM bridge	2 – 4 vCPU, 8 Go, 100 Go	Capacité existante	Infra Talan
VM nœud Sepolia (P2)	4 vCPU, 16 Go, SSD ≥ 1 To	Le stockage est le poste le plus significatif	Infra Talan
Bande passante (P2)	Trafic P2P continu	Plusieurs centaines de Go par mois (à mesurer)	Réseau Talan
SepoliaETH	Gas des mints Sepolia	0 € (faucets)	—
Gas TELA	Selon configuration du réseau privé	0 € (à confirmer)	TELA
Licences, SaaS, prestataires	—	0 €	—

12 Compétences requises

12.1 Matrice des compétences

La colonne « couverture actuelle » s'appuie sur ce que le démonstrateur met déjà en œuvre ; elle ne constitue pas une évaluation individuelle.

Compétence	Niveau requis	Utilisée pour	Couverture actuelle	Appui
Solidity, EVM, OpenZeppelin, Foundry	Avancé	Contrats, adaptation au token, tests, fuzzing, invariants	PRATIQUÉE démonstrateur	Relecteur
Sécurité des smart contracts (rôles, rejeu, réentrance)	Avancé	Revue, modèle de menaces, choix de plafonds	À APPROFONDIR	Relecteur, cyber
.NET / C#, Nethereum	Confirmé	Relayer, signataire	PRATIQUÉE	—
PostgreSQL transactionnel	Confirmé	Verrous, nonces, reprise ; sauvegarde/restauration	PRATIQUÉE (sauvegarde à acquérir)	Infra
Exploitation de nœuds Ethereum (client d'exécution + consensus, synchro, P2P)	Confirmé	Nœud Sepolia Besu + Teku	À ACQUÉRIR	Victor (Besu)
Besu en réseau privé (QBFT, permissionnement)	Intermédiaire	Intégration TELA, finalité	À APPROFONDIR Besu local	Victor
Linux, Docker Compose, durcissement	Confirmé	VM, conteneurs, cloisonnement	PRATIQUÉE	Infra
Réseau et sécurité périmétrique	Intermédiaire	Matrice des flux, proxy, pare-feu, TLS	À APPROFONDIR	Infra, cyber
Gestion de clés (keystore, Web3Signer, coffre, HSM)	Confirmé	Signataire durci, rotation	À ACQUÉRIR	Cyber
Observabilité (Prometheus, Grafana, Alertmanager)	Intermédiaire	Supervision, alertes	PRATIQUÉE	—
Analyse de risques, modèle de menaces	Intermédiaire	Sécurité, go/no-go	À APPROFONDIR	Cyber
Pilotage de projet	Intermédiaire	Paliers, jalons, arbitrages	À APPROFONDIR	Alain
Cadre réglementaire (MiCA, DORA)	Notions	Délimiter le périmètre « test »	NOTIONS	Juridique (hors projet)

12.2 Profils et montée en compétence

Répartition recommandée

- **Porteur** (alternant) : conception, développement, tests, preuves, documentation, exploitation du bridge en test.
- **Référent TELA** (Victor) : réseau, Besu, droits, déploiements, flux.
- **Cyber Talan** : validation des flux, des secrets et de l'architecture.
- **Relecteur indépendant** : pair développeur Solidity/.NET, distinct de l'auteur.
- **Commanditaire** (Alain) : priorités, go/no-go.

Plan de montée en compétence (≈ 3 JH, inclus dans les lots)

- Documentation Besu « Run Besu and Teku on a testnet » et synchronisation par point de contrôle (M1).
- Documentation Web3Signer : stockage des clés en coffre (M2).
- Session de travail avec Victor sur l'exploitation de TELA (L0).
- Revue croisée des contrats avec un pair (M5).

13 Feuille de route et prochaines étapes

13.1 Jalons et critères de passage

Jalon	Objectif	Critère d'acceptation (vérifiable)	Lots
J0	Livrable 1 validé	Décisions D1 à D6 prises ; rendez-vous avec Victor planifié	—
J1	Cadrage terminé	Questionnaire §13.2 renseigné ; accès TELA de test obtenus ; flux F1, F2-A et F12 validés ; option token choisie ; chiffrage réévalué	L0
J2	Socle prêt	VM dédiée ; campagnes `prove` 18/18 rejouées sur les deux profils locaux, sources figées	L1, L2
J3	Contrats sur testnets	Contrats déployés et appariés sur Sepolia et TELA ; identités et codes vérifiés par le relayer	L3
J4	Premier pont réel	Aller-retour réel Sepolia ⇄ TELA ; réconciliation `balanced` ; preuves scellées (hash des transactions, finalité, commit)	L4 – L6
J5	Go / no-go palier 2	Démonstration à Alain et Victor ; registre des risques révisé	L7
J6	Industrialisé	Nœud Sepolia interne ; Web3Signer + coffre ; restauration testée ; runbooks rejoués ; relecture indépendante soldée	M1 – M6

13.2 Préparer le rendez-vous avec Victor

Questions à poser, regroupées par thème. Les réponses alimentent directement le profil `integration-testnet` et la matrice des flux. Aucune réponse ne doit contenir de secret : les accès s'échangent par le canal prévu par la cyber.

Réseau TELA

1. Quel environnement TELA de test utiliser ? Chain ID et hash du bloc genesis ?
2. Version de Besu, algorithme de consensus (QBFT, IBFT 2.0, Clique), période de bloc, nombre de validateurs ?
3. Forks EVM activés (London, Shanghai, Cancun…) ?
4. Prix du gas (nul ?) et obtention du jeton natif pour les comptes du bridge ?
5. Permissionnement des comptes et des nœuds : procédure pour autoriser le déployeur et l'exécuteur ?
6. Accès RPC : URL, HTTP/WebSocket, TLS, authentification, VPN, filtrage ?
7. Historique servi par le RPC : `eth_getLogs` sur de longues plages, lectures d'état à une hauteur passée ?
8. Fenêtres de maintenance, réinitialisations éventuelles du réseau de test ?

Token « Stablecoin Euro Talan »

9. Adresses sur TELA et sur Sepolia, code source et ABI ?
10. Decimals, frais de transfert, rebase, contrat évolutif (proxy) ?
11. Rôles mint/burn existants, administrateur, possibilité d'accorder un rôle au Bridge ?
12. Offre en circulation sur chaque chaîne et détenteurs : quelle offre de référence ?

Infrastructure et cyber

13. Où héberger la VM du bridge (zone, image système, standard de durcissement) ?
14. Flux autorisés : HTTPS sortant vers des RPC publics ? P2P 30303/9000 à terme ?
15. Coffre de secrets ou HSM existant utilisable ?
16. Supervision centrale, SIEM, SMTP, sauvegarde : que réutiliser ?
17. Miroir interne pour images Docker et paquets ?
18. Processus et délais de validation d'architecture par la cyber ?

Gouvernance

19. Qui opérera le bridge à terme ? Niveau de service attendu (au mieux) ?
20. Qui détient les rôles d'administration et de pause ?

13.3 Décisions attendues et validation

Les décisions D1 à D6 sont détaillées en §1.4. En résumé : **activer le palier 1** de l'étape 2, accepter le modèle « relayer autorisé » pour des tokens de test, trancher Sepolia ou successeur, valider l'accès Sepolia du palier 1, confirmer une disponibilité réaliste et désigner un relecteur.

Commanditaire — Alain
Décision : ☐ go palier 1 ☐ ajustements ☐ no-go
Date et visa :

Référent TELA — Victor
Contraintes infra et cyber prises en compte : ☐ oui ☐ réserves
Date et visa :

Auteur — Matthieu Barraque
Version 1.0 du 30/10/2026
Prochaine révision : fin du cadrage (J1)

A Annexes

A.1 Glossaire

Bridge (pont)	Mécanisme qui fait passer la valeur d'un token d'une blockchain à une autre.
Burn & mint	Le token est détruit (burn) sur la chaîne source et créé (mint) en même quantité sur la chaîne de destination ; l'offre totale est conservée.
Lock & mint	Variante : le token est verrouillé dans un contrat sur la source et un jeton représentatif est minté sur la destination.
ERC20	Standard de token fongible sur les chaînes compatibles Ethereum (EVM).
EVM	Machine virtuelle d'Ethereum ; TELA (Besu) et Sepolia l'implémentent toutes deux.
Sepolia	Réseau de test public d'Ethereum (chain ID 11155111), en preuve d'enjeu ; ses ETH n'ont pas de valeur.
TELA	Talan Enterprise Ledger Architecture : blockchain privée EVM sur nœuds Hyperledger Besu, opérée par Talan.
Besu · Teku	Clients Ethereum open source en Java : Besu pour l'exécution, Teku pour le consensus.
QBFT	Consensus à tolérance byzantine utilisé par les réseaux Besu privés ; finalité immédiate si moins d'un tiers des validateurs est défaillant.
Finalité	Moment où un bloc ne peut plus être annulé selon les règles du réseau. Sur Sepolia, tag `finalized` (≈ 13 min).
Réorganisation	Remplacement de blocs récents par une autre branche ; un événement observé peut disparaître.
Relayer	Service hors chaîne qui observe une chaîne et soumet des transactions sur l'autre.
RPC (JSON-RPC)	Interface d'accès à un nœud : lecture d'état, d'événements, envoi de transactions.
Faucet	Service qui distribue gratuitement des jetons natifs de test pour payer le gas.
Gas	Unité de coût d'exécution d'une transaction, payée en jeton natif de la chaîne.
Nonce	Compteur. Nonce de compte : ordonne les transactions d'un émetteur. Nonce source du Bridge : numérote les burns.
Checkpoint	Prochain bloc à lire par le collecteur, persisté avec les événements du lot.
Multisig	Compte dont les actions exigent plusieurs signatures (par exemple 2 sur 3).
HSM · coffre	Matériel ou logiciel qui protège les clés et signe sans les exposer.
MiCA	Règlement (UE) 2023/1114 sur les marchés de crypto-actifs, qui encadre notamment les jetons de monnaie électronique.
JH	Jour-homme : 7 heures de travail effectif.

A.2 Extraits de code commentés

```
// contracts/src/Bridge.sol — côté source : burn et notification dans la même transaction
function requestTransfer(uint256 destinationChainId, address recipient, uint256 amount)
    external whenNotPaused nonReentrant returns (bytes32 transferId)
{
    Pair memory pair = _pair(destinationChainId);          // paire configurée et chaîne inchangée
    _validateTransfer(recipient, amount);                  // destinataire non nul, 0 < montant ≤ max-
Transfer
    uint256 nonce = nextNonce++;                           // 0, 1, 2… vérifié par le collecteur
    transferId = _transferId(block.chainid, address(this), destinationChainId, pair.remoteBridge,
                             address(token), pair.remoteToken, nonce, amount, recipient);
    token.burnFrom(_msgSender(), amount);                  // dépense l'allowance puis brûle
    emit TransferInitiated(transferId, destinationChainId, _msgSender(), recipient,
                           address(token), pair.remoteToken, pair.remoteBridge, nonce, amount);
}
```

```
// côté destination : contrôles, anti-rejeu, quota, puis mint — atomique
function completeTransfer(uint256 sourceChainId, address sourceBridge, address sourceToken,
                          uint256 nonce, uint256 amount, address recipient)
    external onlyRole(EXECUTOR_ROLE) whenNotPaused nonReentrant
{
    Pair memory pair = _pair(sourceChainId);
    if (pair.remoteBridge ≠ sourceBridge || pair.remoteToken ≠ sourceToken) revert PairMismatch();
    _validateTransfer(recipient, amount);
    bytes32 transferId = _transferId(sourceChainId, sourceBridge, block.chainid, address(this),
                                     sourceToken, address(token), nonce, amount, recipient);
    if (processed[transferId]) revert TransferAlreadyProcessed(transferId);
    uint256 epoch = block.timestamp / epochSeconds;
    uint256 total = mintedInEpoch[epoch] + amount;
    if (total > epochMintCap) revert EpochMintCapExceeded(epoch, total);
    processed[transferId] = true;                          // annulé avec le reste si le mint échoue
    mintedInEpoch[epoch] = total;
    token.mint(recipient, amount);
    emit TransferCompleted(transferId, recipient, amount);
}
```

A.3 Traçabilité des preuves

Preuve	Chemin dans le dépôt	Commit	Résultat	SHA-256
Démonstration livrable 1	`evidence/demo-livrable1-20261030T133958Z/demo.json`	`2f05504`	29 / 29	`0cfb06bb3ed4188a9340bb178237f642955a28069c34bb14ba9619467e109785`
Index de la démonstration	`evidence/demo-livrable1-20261030T133958Z/index.json`	`2f05504`	—	`7b62e11b65f60120b140377a85dbfa6464b7812fc48baad4a3d60dc4dde110dc`
Campagne local-fast	`evidence/20261025T002452Z-local-fast/manifest.json`	`b97c541`	18 / 18	`f4d0a64da04cd7a32e7219f584fc2f87992f3f0f8be464130f54bd5195b70461`
Campagne local-besu	`evidence/20261025T003358Z-local-besu/manifest.json`	`b97c541`	18 / 18	`b9943ee6bceea6196eedd4e0bf74a7908514242ada18659902686139fc5b5d63`
CI GitHub Actions	`evidence/ci-b97c541/index.json`	`b97c541`	succès	`6df219d29ee1f0168b7b464aa0be0533cdb0268b4cbdc225720b67be06501b84`
Capture Grafana	`evidence/finalization-b97c541/grafana-fast-b97c541-delivery.png`	`b97c541`	observation	`b5b404786eba92e68e3ebe9a2203d9b2b767264114a21c29a4b1089e8028c60d`

Preuve	Chemin dans le dépôt	Commit	Résultat	SHA-256
Registre des affirmations	`docs/claims.yaml`	—	26 obsolètes · 2 bloquées	`1ab32cd212837583b9f095ee4c7384d972ca6e41bfac3a05d98a118579c6df95`
Contrat Bridge testé	`contracts/src/Bridge.sol`	`2f05504`	—	`7841ea842bb58e93923992a15852ee6abc8ff5536a7c0068eebf6cf0ca02047b`
Token de test	`contracts/src/BridgeToken.sol`	`2f05504`	—	`ce02efa89b02a711363e59b4d372e130c7deb9ac25d2fcda2901c1bcce7db951`

Le PDF est produit par `livrables/livrable-1/build_pdf.py` à partir des sources HTML du même dossier ; il ne contient aucun secret. Les décisions techniques détaillées sont dans `docs/adr/0001` à `0006`, les limites dans `docs/limites.md` et les prérequis manquants dans `docs/informations-manquantes.md`.

A.4 Sources et références

Consultées le 30 octobre 2026. Certaines pages n'étaient pas accessibles depuis l'environnement d'exécution et ont été lues au travers de résultats de recherche : à revérifier avant toute décision engageante.

Source	Utilisée pour
Ethereum Foundation, « Holesky and Hoodi Testnet Updates », 18/03/2025 — blog.ethereum.org/2025/03/18/hoodi-holesky	Fin de vie attendue de Sepolia au 30/09/2026 et annonce d'un successeur
Cointelegraph, « Ethereum schedules Glamsterdam upgrade on Sepolia for Oct. 6 », 2026 — cointelegraph.com/news/ethereum-glamsterdam-upgrade-sepolia ; 24/7 Wall St., « What Is Glamsterdam, and Why Did Ethereum Just Delay Its Testnet Launch? », 23/10/2026 — 247wallst.com	Activité de Sepolia en octobre 2026 (mise à niveau programmée puis rapportée activée le 6/10)
Ethereum Foundation, « Partial history expiry », 08/07/2025 — blog.ethereum.org/2025/07/08/partial-history-exp	Historique pré-fusion de Sepolia, options d'expiration d'historique de Geth et Besu
Hyperledger Besu, « Run Besu and Teku on a testnet » — besu.hyperledger.org/public-networks/tutorials/besu-teku-testnet	Nœud Sepolia Besu + Teku, synchronisation par point de contrôle
Geth, « Hardware requirements » — geth.ethereum.org/docs/getting-started/hardware-requirements	Ordres de grandeur de ressources et de bande passante
Consensys, Web3Signer — docs.web3signer.consensys.io (dont « Use Web3Signer with HashiCorp Vault »)	Signature distante, licence Apache-2.0, coffre KV v2
Hyperlane, « Deploy Hyperlane » — docs.hyperlane.xyz/docs/guides/chains/deploy-hyperlane	Déploiement sans permission, validateurs et relayer auto-hébergés
pk910, PoWFaucet — github.com/pk910/PoWFaucet	Faucet Sepolia à preuve de travail, sans compte
OpenZeppelin Contracts 5.x — docs.openzeppelin.com/contracts/5.x	ERC20, AccessControl, Pausable, ReentrancyGuard
PostgreSQL 17, « Explicit Locking » — postgresql.org/docs/17/explicit-locking.html	Verrous consultatifs, réservation de nonces
Règlement (UE) 2023/1114 (MiCA) — eur-lex.europa.eu/eli/reg/2023/1114/oj	Cadre des jetons de monnaie électronique (mention, sans analyse juridique)
Règlement (UE) 2022/2554 (DORA) — eur-lex.europa.eu/eli/reg/2022/2554/oj	Référence à examiner selon le périmètre réel (sans analyse juridique)